IOT-DRIVEN ENERGY OPTIMISATION SYSTEM WITH AI-BASED OCCUPANCY AND SECURITY MONITORING

BY

ADEYEMI OLUWASEYI EMMANUEL

210407508

A

PROJECT SUBMITTED TO

THE DEPARTMENT OF SYSTEMS ENGINEERING, UNIVERSITY OF LAGOS, AKOKA IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF BACHELOR OF SCIENCE



(B.Sc. HONORS) DEGREE IN SYSTEMS ENGINEERING

SUPERVISOR: ENGR E. ADEWOYIN

May 2026

DECLARATION

I, Adeyemi Oluwaseyi Emmanuel, declare that I have conducted the research and produced the work presented in this report under the supervision of Engr. E. Adewoyin in the Department of Systems Engineering, University of Lagos. I solemnly affirm that, to the best of my knowledge, no part of this report has been previously submitted here or elsewhere in a previous application for an award of a degree. All sources of information and ideas used in this work have been duly acknowledged and properly cited following academic standards.

……

ADEYEMI OLUWASEYI EMMANUEL

210407508

CERTIFICATION

This is to certify that this project titled **"IOT-DRIVEN ENERGY OPTIMISATION SYSTEM WITH AI-BASED OCCUPANCY AND SECURITY MONITORING"** carried out by ADEYEMI OLUWASEYI EMMANUEL, has been approved for meeting part of the requirements and regulations governing the award of the Bachelor of Science degree in Systems Engineering of the University of Lagos, Akoka, Nigeria.

..... **.....**

ENGR. E. ADEWOYIN **DATE**

PROJECT SUPERVISOR

..... **.....**

Dr. OLAYIWOLA **DATE**

HEAD OF DEPARTMENT

DEDICATION

I dedicate this project to Almighty God the source of my wisdom, strength and understanding; to my loving parents, who sacrificed their time and energy in my upbringing, prayed for me; to my family and friends, who encouraged and believed in me throughout the process of this project; to my supervisor, Engr. E. Adewoyin, for her guidance and patience during this research.

ACKNOWLEDGEMENT

I give all glory and honour to Almighty God who gave me life, wisdom, strength and grace to complete this project and this program of study. I am grateful to my supervisor Engr. E. Adewoyin, for her guidance, constructive criticism, patience and constant encouragement throughout this research. Her direction shaped this work and made it better than I could have made it alone.

I acknowledge the Head of Department, and the lecturers and staff of the Department of Systems Engineering, University of Lagos for the knowledge and training that I acquired during my years in the department. I express my gratitude to my parents Mr. and Mrs. Adeyemi for their love, prayers, financial support, and sacrifice. Also, I would like to express my gratitude to my siblings and all my family for their encouragement and understanding.

To my friends and course mates, for making this journey bearable, with your support, ideas, late nights and laughter. I am grateful for all those people who have helped a good deal in the success of this project and this program, God bless you all.

ABSTRACT

High electricity prices, tiered Service Based Tariff bands and poor internet connectivity is challenging Nigerian households and offices in the efficient use of energy. Manual switching isn't reliable, and motion-based or cloud-based smart home systems are inaccurate at detecting still people, are based on timeouts and don't work if there's no internet. This project introduces an all-in-one energy optimization system powered by AI algorithms for occupancy and security monitoring, which operates within a local network.

The system utilizes a ESP32-S3-CAM as an occupancy sensor, and sends the video to Frigate NVR via go2rtc using an OpenVINO-accelerated SSDLite MobileNetV2 person detection model at 50% confidence level. It is processed on an x86-64 mini-PC with a Home Assistant Operating System (HAOS) as well as the Frigate application. Detection events are passed over MQTT to a YAML automation that turns a Zigbee smart plug on when a person is detected and off after 10 seconds (after the space is clear). The plug communicates voltage, current, power and energy readings to a local dashboard via Zigbee USB dongle and Zigbee2MQTT and a Zigbee2MQTT snapshot provides basic security monitoring. The ESP32 S3-CAM was first tested with the Arduino IDE and then programmed with ESPHome. The results of the tests indicated an overall detection accuracy of 88.75%, while the average value in clear visibility, low light and under partial occlusion was 95%, 85% and 80% respectively, and the average detector-to-actuator latency time was ~ 1.5 s without cloud round trip. Automated control reduced average power consumption by 42.3% (3.701 W versus 6.419 W for manual control) compared to manual control in two sample windows of 11 minutes each. Therefore, low cost, local processing, camera based occupancy detection is a feasible option to motion based and cloud based energy management in Nigeria.

TABLE OF CONTENT

DECLARATION ii

CERTIFICATION iii

DEDICATION iv

ACKNOWLEDGEMENT v

ABSTRACT vi

TABLE OF CONTENT vii

TABLE OF FIGURES ix

TABLE OF TABLES x

CHAPTER ONE 11

INTRODUCTION 11

1.1 Background Of Study 11

1.2 Statement of Problem 14

1.3 Aim and Objectives of Study 15

1.4 Significance of Study 16

1.5 Scope and limitations of the study 17

1.6 Definition of operational terms 19

CHAPTER TWO 22

LITERATURE REVIEW 22

2.1 Preamble 22

2.2 Conceptual Review 23

2.3 Empirical Review 34

CHAPTER THREE 38

METHODOLOGY 38

3.1 Research Approach and Design 38

3.2. Hardware and Justification 45

3.3 Software Components 50

3.4 The Software Integration 63

3.5 System Prototype Design 63

3.6 AI Model and Detection Approach 67

3.7 Decision-Making Logic 68

3.8 Basic Security Monitoring 69

3.9 Performance Evaluation Methods 69

CHAPTER FOUR 71

RESULTS AND DISCUSSION 71

4.1. System Implementation Results 71

4.2. Results of AI-Based Occupancy Detection Testing 72

4.3. System Responsiveness (Latency) Results 73

4.4. Energy Optimization Effectiveness Results 74

4.5. Basic Security Monitoring Results 78

4.6. Dashboard and Data Visualization Results 79

4.7. Discussion of Findings in Relation to Study Objectives 80

4.8. Comparison with Existing Approaches 81

4.9. Limitations Observed During Testing 81

CHAPTER FIVE 83

SUMMARY, CONCLUSION AND RECOMMENDATIONS 83

5.1. Summary of the Study 83

5.2. Conclusion 84

5.3. Contribution to Knowledge 85

5.4. Recommendations 85

5.5. Suggestions for Further Studies 86

REFERENCE 88

TABLE OF FIGURES

Figure 3.0: Block Diagram and description 41

Figure 3.1: Mini PC x86-64 Processor with 128gb SSD, 8gb RAM 46

Figure 3.2: ESP32 S3 -CAM Module 47

Figure 3.3: OV2640 camera sensor 47

Figure 3.4: Zigbee Smart Plug with Energy Monitoring 48

Figure 3.5: Zigbee 3.0 USB Dongle 49

Figure 3.6: Arduino IDE Boards Manager - ESP32 Package (v3.3.4) Installed 51

Figure 3.7: Arduino IDE Board Info Dialog ESP32-CAM 51

Figure 3.8: Arduino IDE -Test Sketch Compiling for ESP32 52

Figure 3.9: HAOS Boot Log - System Evaluation and Autofix Complete 53

Figure 3.10: ESPHome Web Installer - Initial Flash at 16% Progress 64

Figure 3.11: ESPHome - Firmware Build and OTA Upload Success Log 65

Figure 4.4: Chart of the sample comparison of Energy Consumption 75

Figure 4.5: Timestamped image log pane in frigate NVR 78

Figure 4.6: History chart log for project energy 79

TABLE OF TABLES

Table 3.1.1: Description of the Block Diagram Sensing Stage 42

Table 3.1.2: Description of the Block Diagram Processing Stage 43

Table 3.2: Cost of Hardware Used 50

Table 4.2: Sample Occupancy Detection Test Results 72

Table 4.3: Sample End-to-End Latency Breakdown of the Detection-to-Actuation Pipeline. 73

Table 4.4.1.1: Sample Comparison of Energy Consumption Duration 74

Table 4.4.1.2: Sample Comparison Consumption Under Manual and Automated Control 75

Table 4.4.2: Raw Power Readings and Test-Window Durations Used in Table 4.4.1.1 77

CHAPTER ONE

INTRODUCTION

1.1 Background Of Study

In the world today, there is a global surge in the demand for electricity, which is due to the growing population. In Nigeria today, there has been an improvement in the supply of electricity from the grid over the last few years. However, this improvement came with an increase in the electrical tariff and also the segregation and hierarchy in the distribution of electricity supply into bands. The electrical structures of the tariff in Nigeria are created to exhibit both the quality of the service delivered and the cost of supply to the consumers. According to the regulatory framework of the Multi-Year Tariff Order (MYTO), Nigerian consumers are classified into Service-Based Tariff (SBT) bands (A–E). corresponding to the number of hours of electricity received daily. For instance, Band A consumers get a minimum of 20-24 hours of supply per day, while Band B through E consumers get fewer hours of supply per day. This segregation mechanism influences the tariff rate charged. (Akpojedje et al., 2025). Although the framework aims to improve the cost reflectivity and support demand side management by aligning tariffs with actual service quality and costs, consumers, especially those with higher supply costs, find it challenging to optimize their energy usage efficiently, and this results in them paying more tariffs even when the system concept of the prepaid meter is a “Pay-As-You-Go” model. (Fagbohun & Femi-Jemilohun, 2017).

To optimize energy usage, both traditional and smart methods of optimizing electrical energy usage have been employed, and these methods have major downsides. The traditional way of optimizing electrical energy usage includes manually turning off the light switches and other electrical appliances when leaving a room. This method has not been one hundred percent effective because most users tend to forget to manually turn these appliances off, and they keep

running, leading to significant electrical energy wastage. Another method of optimizing electrical energy is the use of smart switches, and the addition of sensors like the millimeter wave sensors, motion sensors, and presence sensors. This method has been highly adopted by most smart home owners due to its higher percentage of energy optimization efficiency. However, it also has its lapses. Most smart home owners use this method of installing smart switches for convenience, and some of these methods rely on a stable internet connection to function properly. Countries that experience internet service downtimes will be affected. Moreover, sensors have a limited range of coverage; installing multiple sensors in a very large room to tell if the room is being occupied could be a hassle sometimes, and it requires different levels of integration. In addition, they wouldn't be able to tell if the occupant has left the room, and they rely on a timeout mechanism.

One of the methods adopted to optimize energy usage is the use of IoT. The Internet of Things is the interconnection of physical devices via the Internet, in order for them to be able to collect and exchange data. There are a variety of IoT devices and their implementation and use. (Manoj Kumar, Krishna Mohan Pandey 2023). The word "internet" in Internet of Things (IoT) does not necessarily represent the use of the World Wide Web, but a server-based or peer-to-peer network. The communication embedded in these networks possesses the ability to send raw data or instructions, analyze commands, receive data, control, and create an ideal connection between nodes in the network. IoT can help make different operations more computable and predictable by collecting and processing a massive amount of data. Integrating IoT into power and energy systems established a promising future in the mitigation of environmental and operational issues relating to energy systems. (Orumwense & Abo-Al-Ez, 2022a). In the context of energy optimization with IoT, the communication protocols in IoT devices play a very important role in enabling the efficiency of data exchange between the devices, sensors, and central hubs. These protocols can be categorized into: application layer protocols (such as

Constrained Application Protocol (CoAP), and Message Queuing Telemetry Transport (MQTT) for data messaging) and lower layer wireless protocols (such as Zigbee, Wi-Fi, Bluetooth for physical connectivity). (Mansour et al., 2023). These protocols are important where low power, reliable and scalable network are required to monitor occupancy, control appliances and also integrate AI base security with minimal energy consumption.

There are many existing IoT energy optimization methods and solutions that are already being implemented in today's world, most of the solution rely heavily on sensor triggers and or automations, sensors such as the passive infrared (PIR) motion sensor, the millimeter wave sensor or the proximity sensors and also the addition of timed schedules with the sensors are used as a method to optimize energy usage. (Shokrollahi et al., 2024). These methods are used to detect occupancy, but they have the tendency to misinterpret presence, especially in large spaces where the sensors' range is limited; they may also fail to detect stationary occupants, a sleeping occupant, or they can prematurely switch off essential systems. These can lead to the user's inconvenience, and they already lack efficiency. Moreover, dependency on cloud connectivity for real-time data processing and control for energy optimization could lead to potential vulnerabilities for users who value data privacy. Additionally, there will be a drastic reduction in efficiency where the internet access is unreliable or laggy, especially in many developing countries. (Poyyamozhi et al., 2024). Hence, there is a need for a more efficient, locally executable intelligence that can accurately and effectively interpret the environmental and occupancy data in real time, make energy optimization decisions in real time, and operate independently with or without internet connectivity.

Existing IoT-based energy management systems primarily depend on motion-triggered automation, cloud-based processing, and conventional occupancy sensing mechanisms. However, these approaches often suffer from inaccurate occupancy interpretation, delayed response time, privacy concerns, and dependence on uninterrupted internet connectivity. In

developing regions such as Nigeria, where internet instability and high electricity costs remain major concerns, these limitations significantly reduce system efficiency and reliability. Furthermore, many existing systems operate as isolated smart automation platforms without integrating intelligent occupancy-aware decision-making and security monitoring into a unified framework. Therefore, there is a need for an intelligent, locally processed, AI-driven energy optimization system capable of accurately detecting occupancy, monitoring security events, and autonomously managing energy consumption with minimal reliance on cloud infrastructure.

This project documents the implementation of an IoT-driven energy optimization system using AI-based occupancy method and security monitoring. This is basically the use of locally connected IoT devices such as Esp 32 cam, USB-TTL serial programmer board, the Zigbee dongle, ZigBee smart plug with the energy monitoring feature. Also deploying machine learning models for occupancy monitoring, and also the integration of a centralized automation hub, which is the Home Assistant Operating System (HAOS), the operating system helps to support local connectivity and also to improve energy efficiency with its robust automation capabilities.

1.2 Statement of Problem

In the world today, the traditional methods of optimizing energy by manually switching off appliances are not reliable because, as humans, we can be very forgetful. This can lead to a potential energy wastage. More also, modern way of using IoT devices such as sensors like the motion sensor, millimeter wave sensor combined with smart switches offered better efficiency but they have limitation due to the limited range, not being able to identify stationary or a sleeping occupant, and also, they rely on timeout and they can prematurely switch off essential systems even while there is an occupant in range.

The implementation of IoT devices for smart home automation in Nigeria are mostly marketed as devices for convenience and luxury rather than for their optimal efficiency. Most smart home users in this country use smart home automation to control lighting, air conditioners, fans, curtains and blinds, and some basic appliances from their mobile devices and smart speakers using voice prompts. Also, most of their functionalities rely heavily on cloud access, which requires a constant internet connection to control something as simple as turning on/off a light bulb.

In addition, the use of an IoT camera base solution offers a better occupancy detection using a visual method, and to detect if there are humanoid figures in the picture frame. This solution offers a more robust, intelligent, context-aware solution and also a more reliable energy management system that can accurately detect occupancy and optimize energy in real time, and operate with little reliance on a constant internet connection. These tend to promote locally executed decision-making, drastically reduce energy wastage, and improve users' comfort.

1.3 Aim and Objectives of Study

Aim

The aim of this research proposal is to design and implement an energy optimization system based on IoT and integrated with AI-based occupancy detection and security surveillance to develop intelligent real time energy management.

Objectives

i. Design and create an IoT-based system of real-time monitoring of electricity energy use.

ii. Introduce smart control and automation systems to minimize unnecessary energy consumption in electrical appliances.

iii. Design a camera-based AI occupancy detection system to make intelligent decisions in energy management.

iv. Design a system architecture that is locally processed and requires minimum reliance on cloud or internet connectivity.

v. Introduce data acquisition, processing and visualization to monitor energy usage and interact with the system.

vi. Evaluate the system performance in terms of energy efficiency, reliability, and response time.

1.4 Significance of Study

This project promotes and contributes to the development and integration of intelligent systems that promote energy efficiency in smart environments. With the use of Internet of Things (IoT) technology combined with Artificial Intelligence (AI), this system provides a combined solution for real-time monitoring, analysis, and optimization of energy usage.

This research promotes benefits to organizations, homeowners, facility managers, and office owners while achieving sustainable goals. Enabling them to have better control over energy consumption by reducing electricity wastage and the cost of operation. The Data generated via this research helps users to make informed decisions, predict energy usage patterns, optimize smarter automations, and also detect abnormal consumption, which is very important in regions with unstable power supply.

This research is relevant to the academic and research community as it demonstrates the real-life application of Internet of Things (IoT) and AI techniques in embedded systems. It contributes to the sustainability and technological advancement by promoting energy efficiency and intelligent solutions. For IoT enthusiasts, engineers, smart home enthusiasts and system

designers, this research serves as a scalable solution that can be extended or optimized for larger projects on a much larger scale.

1.5 Scope and limitations of the study

The scope of study covers:

i. **System Design and Development:** This study involves the design and use of IoT systems to optimize energy management and wastage for home owners, organizations, and offices. This also includes the requisite software and hardware for real-time monitoring.

ii. **Use Interface Development:** This study also covers the deployment of a simple user interface to monitor real-time energy consumption, as well as voltage and current. This involves the use of a robust operating system that supports complex integration and automation, as well as monitoring and optimization.

iii. **IoT Data Communication:** This study involves the use of IoT (Internet of Things) communication protocols to communicate and interact, and also send and receive data from embedded devices to a central processing system or platform.

iv. **Prototype Implementation:** The study is also involved with the development and testing of a functional prototype that can be implemented and is also realistically suitable for residential, office, and organizational use.

v. **Energy Monitoring:** This project features the monitoring of electrical energy consumption of the appliance using smart plugs with energy, voltage, and current monitoring. This will be used to collect real-time data.

vi. **AI-Related Data Analysis:** This research includes the application of basic machine learning techniques and artificial intelligence to analyze the generated and collected data and generate insights from the data. Insight, such as energy usage.

vii. **Occupancy Detection:** This research includes the use of images via an IoT camera to detect the occupancy status in a space for energy optimization. This will be used to trigger the ON/OFF action of the energy monitoring smart plug.

The Limitations of the Study Include:

i. **Implementation of Prototype:** Based on this research, this system is built as a prototype for demonstration. Although some of the hardware can be realistically implemented, the major hardware component used for occupancy detection cannot be used for long term solution. Hence, the prototype is for research and academic purposes.

ii. **Limited Development Scale:** This project is designed for a limited scale smart environment and not for a very large-scale application.

iii. **Embedded Hardware Restrictions:** For the hardware, the memory capacity, the storage, the connectivity range, and the speed of the devices tend to limit the complexity and efficiency of the algorithm.

iv. **Dependency on Internet Connectivity:** Features such as remote monitoring, real time data transfer, and transmission depend on internet availability, which can be unstable in some locations of implementation.

v. **Power Supply Issues:** There is a major challenge of inconsistent power supply; therefore, a power outage can interrupt the system operation. The prototype heavily relies on a constant power supply and does not include a backup power solution. Also, voltage fluctuation can interrupt the system operation, and this will significantly affect continuous data collection from the system.

vi. **Basic Artificial Intelligence Implementation:** This AI component is limited to basic data analysis and pattern recognition, as we rely on an Add-on installed within the operating system. Hence, accuracy is not guaranteed.

vii. **Restricted Data Storage Capacity:** We have a limited storage space of 128gb within the Operating system. With this, long term historical data storage is limited due to the dependency on limited local memory.

viii. **Limited Testing Conditions:** The prototype was implemented and tested in a limited and controlled space. And the outcome can differ in a real-world deployment situation.

1.6 Definition of operational terms

Energy Optimization: Energy Optimization is defined as the process of controlling and managing electrical energy consumption. It is a required process to reduce energy wastage, minimize service cost, and also improve efficiency while retaining the user system performance.

Internet of Things (IoT): This is referred to as a communication network that includes interconnected hardware devices that are embedded with sensors, pneumatics, actuators, with a communication capability that allows them to exchange, collect, and process data either locally or via cloud for control and monitoring.

Smart Home Automation: Smart Home automation can be defined as the use of IoT devices to automate electrical home systems, appliances, and other loads.

Occupancy Detection: This is the process of determining if someone or something is present or absent within a space. This process uses intelligent methods such as millimeter wave systems, camera-based vision systems, and essential algorithms to enable automated decision-making.

Artificial Intelligence (AI): Artificial Intelligence is the use of computational techniques and methods in a way that mimics human intelligence to enable systems to perform complex tasks like recognition of patterns, data analysis, and decision making.

Local Processing: Local processing is the method whereby tasks such as decision making, data collection, data optimization, integration, and automation are done within the local network or a device without relying on cloud or the internet.

Machine Learning: This is a process that enables a system to learn from data, in order to improve decision-making without being programmed for every scenario. This process is a subset of artificial intelligence.

Home Assistant Operating System (HAOS): This is a centralized and open-source operating system that serves as an automation platform. It is used in this project to process data locally with little to no reliance on the cloud or internet, to manage IoT devices, carry out automation rules, and also provide a user interface for monitoring and control of the whole system.

Energy Monitoring: This is the real-time recording and measurement of electrical parameters such as voltage, current, power, and energy usage.

Smart Plug: This is an IoT device that is used to control the power supplied to the electrical appliances connected to this plug. It has the energy monitoring capability. It provides real-time energy consumption and usage data.

Zigbee Protocol: The Zigbee protocol is a short-range and low-power wireless communication protocol. It is used in this project to enable efficient communication and data transfer between IoT devices and the central automation, relying on the internet.

Cloud Dependency: Cloud dependency is the reliance on internet-based servers for data storage, data processing, and the control of IoT devices. In this project, it will affect the system reliability, especially when there is unstable internet connectivity.

Add-ons: These are additional standalone applications, run as Docker containers that enable users to add more services and tools without manually setting up Docker or servers.

Prototype System: A prototype system is a limited but functional implementation of a system design that is developed primarily for demonstration, testing, and academic purposes

Energy Wastage: Energy wastage is the inefficient, unnecessary, or excessive use of energy. It is when excess energy is consumed that is required due to poor automation or negligence, resulting in a higher electricity bill.

Real-Time Monitoring: This is the continuous display and observation of units used or consumed, and also the system status, with a little delay between data visualization and generation.

Embedded System: This is a computing system specially designed to perform a specific task within a larger device. It is a programmable system, that is, it can be programmed once with firmware, and it runs a dedicated task.

CHAPTER TWO

LITERATURE REVIEW

2.1 Preamble

The integration of Internet of Things devices and set-up of these devices with monitoring systems seems to directly be the new way to track the energy consumption across homes, offices, and business buildings. IoT Sensors offer real time statistics and improving the insight into daily power consumption as well. This enhancement and strategy result in faster automated reactions. Systems rather than receiving delayed responses carry out quick adjustment according to the altering usage pattern. Such systems are frequently hybrid, with wireless connectivity, cloud-based systems, and learning systems, and it is possible to have trends developing, without latency. Such operations will manage to reduce energy wastage, continuous/conserved and power provision, and long-term objectives associated with a reduced environmental impact by converting raw signals/data into definitive insights.

It has evolved very fast with overtime in the development of smart systems. One of the ways, that smart homes started working with internet was increasing the scope of using devices. Energy networks also changed quickly. They even instituted some places that had solar systems that could be adjusted. It was better to forecast when machines were trained with examples of real-world events. Glitches became the norm as systems were kept a close eye on themselves. There were controls that were adjusted seconds before problems occurred. Scientists went on to associate sensor measurements with prediction algorithms with advances made in experimental activities conducted both in laboratories and cities throughout the board. (Suwarjono & Afriza, 2025).

To bring out the subject matter, it begins with a discussion of the fundamental concept of the topic with an extensive analysis of articles and journals that have been conducted by the researcher. This review will critically evaluate their studies, their designs, finding and weakness

of the studies. This review will determine the gaps in the approaches and the problem identified which will enable justifying the proposed intervention solution using the approaches.

2.2 Conceptual Review

2.2.1 Internet of Things in Energy Management

Radio-frequency identification (RFID), smart sensors, communication networks, and Internet protocols are the new technologies that have turned the Internet of Things (IoT) into a game changer. Going to the core of the matter, IoT enables interconnected devices to perceive, communicate as well as react to environmental information with minimal human assistance. This would be particularly handy in systems that have to manage energy, where continuous monitoring and auto-fallout would be essential to curtail the energy consumption, enhance functionality as well as increase reliability of the system. The IoT technologies have been accelerated by the development of Internet infrastructure, mobile computing, and machine-to-machine communication.(Al-Fuqaha et al., 2015). IoT devices can detect the trends of usage, locate issues, and contribute to prediction when using analytics and gathering the real-time information. The further evolution of IoT technologies makes it possible to more complicated approaches in automation by connecting physical systems to digital intelligence as a result of connecting technologies to sophisticated computing frameworks. The IoT-based energy management systems of the modern world are designed according to a stratified architecture that incorporates sensing, communication, middleware, and analytics engines. These layers collaborate in order to facilitate a seamless data collection, transferring, processing as well as control. The interaction between these components is also important to know in order to make scalable and compatible systems. An in-depth examination of IoT protocols and architectural concerns assists researchers and developers to devise effective solutions with no need to address complicated standards themselves. IoT in energy management has a number of challenges despite its potential. (Al-Fuqaha et al., 2015). These are problems in interoperability, security

threats, issues of scalability and data management complexities. To resolve these challenges, it is necessary to integrate IoT to new technologies such as cloud computing, fog/edge computing, and big data analytics. These beneficial technologies enhance system responsiveness, reduce delays and facilitate improved decision making within the networks of distributed energy. Practical examples of IoT energy management systems show how communication standards and intelligent control can be used to optimize energy consumption in intelligent residences, buildings and industrial environments. These lay emphasis on the fact that integrating systems well is needed as an avenue to ensuring reliable, efficient, and scalable energy solutions. (Al-Fuqaha et al., 2015).

The IoT-based energy management has attracted substantial attention due to the growing global energy consumption and the necessity to get more sustainable energy. The buildings consume a significant proportion of world energy, so they are also important in terms of their efficiency enhancement. It has been found out that with proper utilization of new monitoring and control technologies building energy consumption can be reduced by up to 3080 percent. IoT-based systems can be used to meet this objective by providing an in-time electrical loads monitoring. This enables facility managers to observe the usage trend and applies evidence-based strategy to maximized energy consumption. (Bhatta et al., 2024a).

An Energy Management System (EMS) that is based on the Internet of Things operates as an automation system. It collects the electrical readings with different sensors and smart meters, processes it, and shows it in visualization platforms. This is useful in coming up with informed decision. Old energy data generated by these systems makes it possible to provide more advanced analytics, such as the identification of peak demand, anomaly detection, and prediction of future requirements. These characteristics assist in reducing the operations expenses as well as enhancing the use of resources. IoT based EMS solutions are simple and

cheap unlike the traditional system of building management which may be expensive and intrinsic, and thus they can serve small and medium size buildings.

The systems of IoT energy monitoring are different phases. This is composed of power layer, data acquisition layer, communication network layer and application layer. The electrical loads and measurement devices are found in the power layer including smart meters. The data acquisition layer records the actual electrical parameters in real-time and then transmits them over networks of communication with communication protocols such as MQTT. Data storage, visualization, and analytics tools at the application layer will give helpful insights regarding the use of energy. This architecture promotes compatibility, and modularity in smart energy structures. (Bhatta et al., 2024a). Environmental and electrical sensors, automated control actuators, data filtering and analysis processing units, and communication networks are some of the important functional components of IoT ecosystems that facilitate the free flow of information. Using the combination of these factors, autonomous monitoring and responsive control is possible, and this would help in rational optimization of energy. Other IoT architectural platforms such as three-layer, cloud-based, and fog enabled system architectural models are used to enhance IoT system performance, security, and scalability in the application of energy systems. (Bhatta et al., 2024a).

Additionally, the IoT-based energy management system favors elaborated prediction algorithms as they generate high-resolution data. The future energy demand can be forecasted with the help of statistical and machine learning models, including autoregressive moving average (ARMA). Planning, load balancing and decision making are improved by forecast and particularly in institutions or campuses where the trends of energy use are unpredictable.

Lately, the developments of systems of energy management based on the IoT have proven the significance of real-time surveillance in enhancing the efficiency of operations in buildings and

smart environments. IoT infrastructure enhance distributed sensing with smart meters and embedded sensors that constantly measure electrical parameters of electrical energy, voltage, and current. This real time visibility enables the facility management to understand the areas that are inefficient, areas that are at optimal use as well as to carry out responsive optimization measures. Research indicates that structures constitute a good percentage of total energy use in the world, and incorporation of IoT surveillance technologies can significantly cut back on the wastage by use of data-based control measures. (Orumwense & Abo-Al-Ez, 2022b).

The architectural design of modern IoT energy systems is usually designed by the application, communication, and sensing layers. The sensing layer is concerned with obtaining the correct consumption data, whereas communication layers follow the lightweight protocols to provide the reliability of data transfer. On the application phase, raw measurements or data are converted into actionable insights using visualization dashboards and analytics platforms and tools. This modular design improves on scalability and compatibility resulting in easy integration with smart optimization frameworks. (Hossein Motlagh et al., 2020). This design offers a robust basis of higher automation in which energy supply may be modified on an operation-by-operation basis. One of the major changes in the IoT-based energy management is the shift in passive monitoring to smart and context-driven optimization. The sequences of continuous data provided by IoT devices enable the systems to identify consumption patterns and update control strategies in response to it. Combined with occupancy detection, energy monitoring enables buildings to match energy consumption with the actual human presence and occupancy level. This reduces the amount of energy wasted when idle and enhances enthusiasm in the overall performance of the system, which underlines the significance of incorporating sensing smartness into the energy optimization systems. (Orumwense & Abo-Al-Ez, 2022b).

Moreover, IoT systems are useful in complete predictive analytics based on the gathering of high-resolution historical datasets. Models of forecasting used on these samples make it

possible to plan ahead, manage demands and schedule in an automatic way. As opposed to responding to energy use, when it happens, intelligent IoTs use forecasting algorithm to predict future patterns of use and pre-set operational limits. The given predictive power is especially useful in intelligent buildings, in which occupancy-based energy optimization will demand constant adaptability to the current usage circumstances. (Orumwense & Abo-Al-Ez, 2022b).

The features of cost-effectiveness and scalability of the IoT are the key benefits of the energy management solution based on IoT. IoT implementations are based on low-cost hardware, wireless connections, and open-source architecture compared to classic building management systems so that they can be deployed with ease. (Hossein Motlagh et al., 2020).

2.2.2 AI in Smart Homes or Smart Buildings

Farzaneh et al. (2021) undertook an in-depth analysis of recent studies on the use of the artificial intelligence (AI) in smart buildings concentrating on the building management system (BMS) and demand response program (DRP). Their research focused on the role of AI technologies in enhancing the control of energy, reliability of the functioning process, and automation of smart environments. Other critical open issues and future research directions were also given by the authors, as they suggested an evaluation framework to determine progress in the major AI fields such as energy efficiency, occupant comfort, building design, and maintenance. They also report that bringing AI to sensor networks and big data analytics makes the decision-making processes smarter than ever before and helps to use less energy, but on the other hand, leads to the increased performance of buildings.

The article also indicated that simulations and predictive models based on AI can optimize the work of a house learning how it is used and changing the parameters of the systems. This kind of smart control mechanism is better to make occupants more comfortable and reduce the waste of energy. This integrating helps the value of AI as a facilitator of the next-generation smart

building systems that promotes and enable productivity, automation, and usability (Farzaneh et al., 2021).

Equally, Almusaed, Yitmen, and Almssad (2023) investigated using AI models in strengthening the design and functional functionality of smart homes, especially in living rooms. Through their activity, it is possible to see how AIs-based systems can enhance environmental regulation, safety, and comfort and can encourage energy efficiency. Integration of smart systems into home infrastructure through intelligent decision-making allows dealing with flexible reactions to user actions and environmental parameters, which results in optimization of energy consumption and enhancement of residential comfort.

Continuing this view, Kanimozhi and Sneha (2024) studied the dual aspect of artificial intelligence in the smart home setting, highlighting the aspect of technological progress and new avenues of security issues. Their paper brings out the aspect of the AI-based smart systems going beyond automation to provide proactive and contextual informed decision-making. The authors talk about the weaknesses of the virtual assistants and smart aging system and demonstrate the need to deal with the problem of security and privacy when more and more people are willing to use AI. Another important contribution of their work is the development of the concept of the so-called Ambient Intelligence (AmI) where intelligent environments react in harmony to the presence and behavior of their users. Building on this point, the authors provide a new Home Intelligence (HI) model that can support proactive smart homes that can predict user demands. These smart systems make use of real-time sensing and active learning to promote operational efficiency and also occupant comfort.

Moreover, the research examines the distance between the state-of-the-art AI researches and the products of the smart home offered commercially. Research has shown that AI-driven optimization also has a lot of potential but practical application is frequently constrained by

issues of scalability, integration and reliability. According to the authors, the smart building ecosystems of the future should maintain a balance between energy efficiency, the comfort of users, and system security to enjoy viable implementation. This view supports the increased role of AI as a key motivating factor behind the emergence of smart, occupancy-aware space that underpins the efforts to manage energy resources in a sustainable way (Kanimozhi & Sneha, 2024).

These studies point to the fact that the implementation of AI in intelligent buildings promotes intelligent automation and the making of decisions driven by occupancy, and optimization of energy. It can be correlated to the ever-expanding vision of intelligent spaces wherein AI-based systems constantly learn, gain adaptability and streamline the processes in a building to attain sustainability and improved user experience.

2.2.3 Occupancy Detection Techniques

Occupancy detection is a key part of a smart building as in real-time it gives information on the movement and presence of occupants of the building that can be used to intelligently control occupancy-controlled systems like lighting, heating, ventilation, and air conditioning. The article by Chaudhari et al. (2024) provides an in depth review of the principles, methods, and technologies behind occupancy detection in smart spaces with the use of IoT sensors. It brings to light how the occupancy information can facilitate energy optimization via the automation of building systems that only work upon the presence and absence of occupants. The authors review the conventional and machine learning-based approaches to detection, explain the pros and cons of each, and address the most significant aspects involved in the detection performance, such as sensor position, data quality, and environmental interference. The increased concern on the development of more effective occupancy detection solutions integrating multiple sensor types and smart algorithms to enhance accuracy and flexibility in the dynamic building environment is also highlighted by this work. (Chaudhari et al., 2024).

Besides sensor-based methods and machine learning techniques, there are also device-free methods of occupancy detection which have attracted a lot of attention to the research of smart buildings. Yang et al. (2018) presented their occupancy sensing framework that was designed and implemented in real-time and did not need any gadgets, addressed the concept of using Wi-Fi devices as ubiquitous wireless signals for human presence sensing, which uses Wi-Fi equipment and devices without creating a dedicated intrusion camera setup. This method embraces the changes in the nature of Wi-Fi signals with human motion to determine occupancy rates, which is a privacy-safeguarding approach compared to the conventional ones. These types of systems are especially useful in smart houses and constructions where occupant privacy and minimization of infrastructure costs are some of the key considerations and accurately and responsively controlling energy. Automated control of lighting, HVAC, and other systems that require energy through the adoption of device-free sensing in the IoT is able to improve energy efficiency and resource availability through the provision of real-time occupancy data.

2.2.4 Smart Security Monitoring System

Smart homes and buildings use security monitoring, which is a fundamental part of intelligent environments, especially due to the fact that the spread of the IoT-based devices will make the interconnected networks more complex and vulnerable. A data-driven security monitoring system that employs analytics based on the IoT device data to identify and stop possible security threats in their tracks was proposed in the study by Al-smaysim and Akbari Bengar (2024). The framework leverages the information produced by the interlinked sensors to detect abnormal behavior and notify users through the use of mobile devices to boost the safety and awareness of the situational context in the smart residential environment. In this regard, encryption and anonymization measures were implemented in order to resolve the privacy and data security issues, and guarantee that confidential data is not compromised during the process of monitoring. (Al-smaysim & Akbari Bengar, 2024) tested the suggested system on the basis of

both simulated and realistic scenarios proving that it is efficient in detecting and responding to various security incidents. Such a study will become part of the creation of proactive and smart security solutions, which are a crucial component of integrated smart home and smart building systems.

The paper by Fitriyan and Syafii (2024) included the design and development of an IoT-based smart home monitoring system that will have additional security features to enable remote and automatic control of the home environment. Their contributions revolve around a web-enabled surveillance arch that is based on a Raspberry Pi microcontroller and various sensors to oversee and regulate electronic devices and also to combine security features that keep users secure and private. The provided system allows one to effectively monitor what is happening in the home and react to the things through Web-based interfaces, which illustrates how IoT frameworks can help not only enhance convenience but also become a formidable force in assuring the security monitoring in residential surroundings. The current study will bring up the dual nature of the IoT connection in smart homes, allowing to control the environment and appliances in real-time and ensuring security mechanisms to prevent unauthorized access and the occurrence of threats.

Expanding on the rationale behind this strategy, additional research has also explored intelligent security surveillance systems, where real-time analytics and pattern recognition are employed to detect illicit activities which predict an intrusion or a break in into systems. Such systems will normally integrate sensor data with adaptive algorithms to make the detection more accurate and responsive. Blending of these IoT-coordinated safety apparatuses with more comprehensive smart building functionalities (e.g., energy services and occupancy checking) is another example of how goal safety, comfort, and efficiency converge in intelligent built environment nowadays. (Fitriyan & Syafii, 2024).

Besides more recent IoT-based systems, some of the underlying research in the past has played part of the leading work toward the establishment of the intelligent security surveillance in smart homes. Dahmen et al. (2017) proposed an activity-based security monitoring framework involving an activity-based learning approach that is based on embedded sensor networks and machine learning algorithms to detect and learn typical behavior in smart home settings and occupant behavior. Their model is a model of normal human activities based on the data gathered with pervasive sensors and any deviation that can be viewed as a possible threat or an abnormal event. Through the assessment of the system implemented on the actual smart home test beds, the research paper revealed that activity-aware analytics can enhance the reactiveness and precision of the security monitoring, and how intelligent threat detections can be achieved through integrating behavioral data with IoT sensor data. In this study, the authors underline the importance of machine learning in the process of improving security services in smart homes and demonstrate that the context-based situational awareness of advanced analytics could be used to supplement conventional alarm systems. (Dahmen et al., 2017).

2.2.5 Related Architectures and Technologies

The article by (Elkhoukhi et al., 2025) has given an excellent overview of the emerging technologies in sensing and data processing that facilitate smart building services and the article brings a lot of useful insights in the architectural underpinning of an efficient and adaptive intelligent environment. Of particular importance regarding their review is that the Internet of Things (IoT) and big data technologies must be integrated to collect, transmit, and process large amounts of heterogeneous building data that is generated by the environmental sensors, occupancy sensors, and energy systems. The authors emphasize the role of developed data processing frameworks based on such platforms as Apache Kafka or Apache Storm or Apache SAMOA as the very essence of high-performance IoT architectures that can deal with the major challenges of interoperability, scalability, real-time analytics, and fault tolerance. To support

this, they state that a layered architecture which includes sensing, network, data processing and application functions permits the creation of intelligent platforms which can support a variety of services, such as occupancy detection, comfort management, anomaly detection, and energy optimization. Moreover, the offered architecture provides an example of how these technologies may be used to ensure successful decision-making and control in real-time over the functioning of the building, therefore, increasing its energy efficiency and comfort of the occupants. Implementing new methods such as big data stream processing and real-time analytics in the environment of IoT offers an expansionary technological foundation, which is needed by next-generation smart building initiatives.

To supplement large-scale architectural structures, (Bhatta et al., 2024b) reviewed how an IoT-based architecture of energy monitoring can be implemented to optimize the processes of buildings by real-time sensing and smart data management. Their article illustrates how multi-layered IoT infrastructure comprising of sensing systems, communicational systems, cloud systems, and visualization systems may be designed in a way that enables constant monitoring of energy usage and at the same time adaptive decision making. Interoperability between hardware sensors and software platforms is also emphasized in the study as it will allow the acquisition of data and the control of the remote systems. The proposed architecture is scalable and can process data to provide actionable insights on the optimization of energy, by incorporating web-based dashboards and cloud storage. The authors also reiterate that these types of modular architecture are needed to facilitate applications in smart building where the occupancy patterns, device usage and environmental conditions are to be managed as a whole. This study supports the value of hierarchical IoT schemes towards providing dependable, dependable, and energy-conscious intelligent building systems that resonate far better with smart usage-based optimization structures.

2.3 Empirical Review

The incorporation of the Internet of Things (IoT) into the smart energy systems is a topic that has been actively studied in recent empirical studies thanks to its positive effects on energy efficiency, better monitoring of the systems, and advanced automation. One of the most interesting studies on the topic is the article by (Orumwense & Abo-Al-Ez, 2022a) which is titled Internet of Things, Smart Energy Systems: A Review on its Applications, Challenges, and Future Trends, which thoroughly analyzes the use of IoT technologies in different energy areas, including smart homes, buildings, industries, and cities, both in terms of application and implementation obstacles. The authors, in their findings, also note that IoT makes it easier to communicate between incompatible energy subsystems, helps optimize energy consumption, and achieves less environmental impact due to better integration of renewable energy sources; the authors, though, also outline considerable obstacles such as communication interoperability, security weaknesses, and scalability problems all of which continue to be present in actual application.

In a recent empirical study, (Bhatta et al., 2024a) designed and tested an IoT-based energy monitoring system that would optimize the building operations to understand that buildings are one of the major consumers of energy worldwide and they need to be managed in an efficient way to ensure the experiences of lowering costs and impact on the environment. They were engaged in the implementation of the IoT sensors, smart meters, and communication networks to gather as much data on the energy consumption of buildings as possible, and then interpret it through the use of statistical and data-based methods to understand peak and off-peak demand patterns and inform energy-saving strategies. The system proved to be practically feasible in real time monitoring and analysis, and being a more affordable system compared to the conventional commercial energy management system, particularly in small to medium-sized buildings. Although these are practical benefits, (Bhatta et al., 2024a) also found that the

scalability was limited and the development of the predictive machine learning models to predict the energy demand proactively and provide automation beyond the descriptive analytics was still required.

The empirical findings by (Farzaneh et al., 2021) titled Artificial Intelligence Evolution in Smart Buildings to Energy Efficiency indicates the increasing presence of intelligent systems in improving energy consumption and performance of the building. (Farzaneh et al., 2021) provided experimental assessments of AI enhanced IoT systems comprising sensor networks and machine learning algorithms to provide automatic control of HVAC, lighting and occupancy-based control and showed notable gains in energy consumption reduction without compromising occupant comfort. The study indicates that dynamic control systems based on AI can change the behavior of the system based on real-time environmental information and decrease the total energy consumption and peak load through predictive and self-learning control. Alongside these encouraging findings, the research also found out a set of ongoing issues concerning data privacy, cybersecurity risks associated with interdependent IoT systems, as well as the difficulties associated with the implementation of such integrated solutions on a large scale and across different types of buildings.

Another interesting empirical study conducted by (Chaudhari et al., 2024) is the study of the fundamentals, algorithms, and technologies of occupancy detection in smart buildings through the use of IoT sensors, where the authors emphasize the importance of the technology in energy consumption and security monitoring. The assessed literature encompasses a wide range of the latest occupancy detection methods of both the classical sensor-based approach and the sophisticated machine learning-based methods, illustrating how occupancy data in real-time can be used to automatic the lighting, heating, ventilation, and air conditioning (HVAC), and also access control systems. (Chaudhari et al., 2024) has reiterated that proper occupancy detection can achieve significant energy savings through proper operation of building systems

where and when required and also provides greater security because of recognition of unusual or unexpected occupancy patterns that might be indicators of intrusions or anomalies. Nevertheless, the authors indicate that there are still issues like the effect of sensor placement, the accuracy of real-time data, privacy, and interference with nature, which opens more opportunities to conduct a further study on the development of resilient and contextually adapting occupancy detection systems.

According to (Al-smaysim & Akbari Bengar, 2024) the study adopted a smart system with the ability to collect and process data on IoT devices and generate information about possible security violations and inform users via an Android-based app. The system was experimented in both simulated and realistic conditions, as it was capable of identifying and addressing a broad scope of the potential security risks through encryption procedures to secure the confidentiality of sensitive data. Their results suggest that information-driven security analytics may play a vital role in enhancing situational awareness and responsiveness of connected residential settings, although there is still a problem of striking a balance between real-time processing and performance and privacy demands.

A study by (Dahmen et al., 2017) proposed an activity conscious security monitoring system of smart homes that makes use of sensor networks and machine learning-based activity learning to identify anomalies that may be used as signs of a possible security risk. (Dahmen et al., 2017) solution was deployed in the CASAS smart home platform (Center for Advanced Studies in Adaptive Systems), where sensor data like motion, door activities (open/close) and ambient environmental data was recorded and analyzed using activity recognition functions to form models of normal resident behaviour and consequently detect abnormalities that can indicate intrusion by security personnel. Experimental testing carried out on various smart home testbeds has shown that the learning of activities was effective in identification of unusual patterns, which could be used to timely respond to automated responses and generally increase

situational awareness. However, the paper also pointed out the difficulties concerning data preprocessing, model generalization to new dwelling settings, and the fact that the continuous learning process is necessary to keep up with the changing behaviour of the occupants.

In conclusion, the reviewed literature provides a strong basis for understanding IoT-driven energy optimization and AI-based monitoring systems, while also highlighting key research gaps. Studies show that combining IoT and AI technologies improves energy efficiency, enables real-time monitoring, and allows for automated control in smart homes and buildings. However, several challenges remain: issues with device compatibility, limits on scalability, data privacy concerns, and cybersecurity risks still stop widespread use. Additionally, traditional methods for detecting occupancy through sensors face significant challenges in identifying stationary occupants; they often depend on timeout mechanisms that can turn off important systems too soon.

These gaps support and explain the current study. This research tackles these shortcomings by suggesting an IoT-driven energy optimization system that merges AI-based occupancy detection through camera vision with thorough security monitoring. By using locally executed intelligence through the Home Assistant Operating System, the system can function without relying on cloud connectivity, which addresses reliability issues in areas with poor internet. The system also incorporates energy monitoring smart plugs with Zigbee communication for tracking real-time consumption while keeping local control, which addresses challenges in scalability and device compatibility. Moreover, using machine learning models with the ESP32-CAM module provides a solid alternative to traditional sensors, avoiding issues with detecting stationary occupants and lowering the risk of accidental system shutdowns. This study thus offers a practical, context-aware method that balances energy efficiency, security monitoring, and user comfort while reducing reliance on external cloud services.

CHAPTER THREE

METHODOLOGY

3.1 Research Approach and Design

3.1.1 Overview

This project implores a Design Science Research (DSR) approach to create a system that optimizes energy with the use of AI-based occupancy detection and monitoring and also generating data on power consumption, energy, voltage and current of the load. The core idea as stated in the objective is to develop and design a working prototype of an IoT system for real-time monitoring of energy consumption while also reduce energy wastage using intelligent control and automation of electrical appliances. The design integrates a user-friendly interface for real-time visualization, control, and monitoring of the system that does not rely heavily on internet/cloud connectivity to operate.

At the initial stage, the process starts with the identification of the system requirements, by determining what needs to be considered and measured and what the outputs (smart automation of electrical load, real-time monitoring of energy usage) are expected. The input layers of the system include the sensors such as the ESP32 S3 camera for human detection and an electrical smart plug sensor for measuring voltage, current, and energy consumption. The system is designed using an automation-based operating system installed on a mini-PC with a 128 solid state drive and 8 gigabytes of RAM (Random Access Memory). This operating system is the host or the brain of the whole system. It is responsible for hosting the open-source NVR (Network Video Recorder) which is configured and built for real-time AI object detection locally. The Operating system also provides a local IP address which displays the user interface and also capture data such as the voltage, current, energy consumption from the smart IoT device. The communication that happens between the components of the whole system is made possible with the aid of an IoT protocol called the MQTT (Message Queuing Telemetry

Transport), this is a lightweight network protocol that is designed for machine-to-machine communication. The MQTT ensures that there is efficient and reliable data transfer within the system.

Phase 1 (Identification and collection of hardware): For this prototype to be realistic, using a mini-PC, with an 8gb DDR4 RAM and a 128gb SSD storage is ideal rather than using a Raspberry Pi 4 because, the whole the system integration has to be reliable and works locally with internet connection. Also, the AI model for identification too will work locally, and it requires high CPU & GPU usage. A raspberry pi would not be able to handle this model. The model can result to the raspberry pi system crashing. Also considering cost, a Raspberry Pi 4 cost about #150,000 on average, a used mini-Pc with an 8gb RAM, 128gb SSD, core i7 intel processor 6th generation cost about #120,000. This is capable of handling the local AI detection model.

Other hardware used are the ESP32 S3-CAM with an embedded type C port for connectivity and power. Along with it is the OV2640 Camera Module for ESP32 CAM Camera Module 2MP with 160 Degree angle view and 650nm lens. For the ZigBee hardware, we have the Zigbee 3.0 USB Dongle Plus Stick with Antenna Compatible. And the ZigBee Smart Plug 16A with Power Monitor Function.

Other essential hardwares are four 3mm wide LEDs for the camera flashlight in lowlight conditions, four 100 ohms resistors, two 3.7v lithium-ion batteries to power the camera and the LEDs, HW-681 6-26VDC to 5VDC 3A DC-DC Buck module, 18650 2S Battery Casing for 3.7v lithium-ion batteries.

Phase 2 (Installing Home assistant Operating System): The home assistant operating system was installed on the mini-PC through a Linux operating system distribution (Ubuntu). Home assistant operating system cannot be directed installed on a dedicated Generic x86-64 PC. It is

more of a complex method to do that. The medium use in this project was that we Boot Ubuntu from a USB flash drive and install the Home Assistant Operating System from there. It also works on laptops and PCs with internal hard disks.

Phase 3 (installing Add-ons): After the installation of the HAOS, we then install the required add-ons needed for this project such as the frigate, ESP-home builder, MQTT-Mosquito. It is with this add-ons were able to deploy the computer vision algorithm, configure the esp32 camera and configure the ZigBee devices and dongle.

Phase 4 (Testing and Automation): After the configuration**s**, testing the system and the automation using a YAML (Yet Another Markup Language) code. The code takes the information from the camera on the frigate add-on via the MQTT, then converts it into action i.e., either to turn on or off the Zigbee smart plug with is also done via the Zigbee2MQTT protocol. Simultaneously, the Zigbee smart plug provides the Energy, voltage, current and power information on the HOAS dashboard via Zigbee2MQTT protocol.

3.1.2. Block Diagrams and description

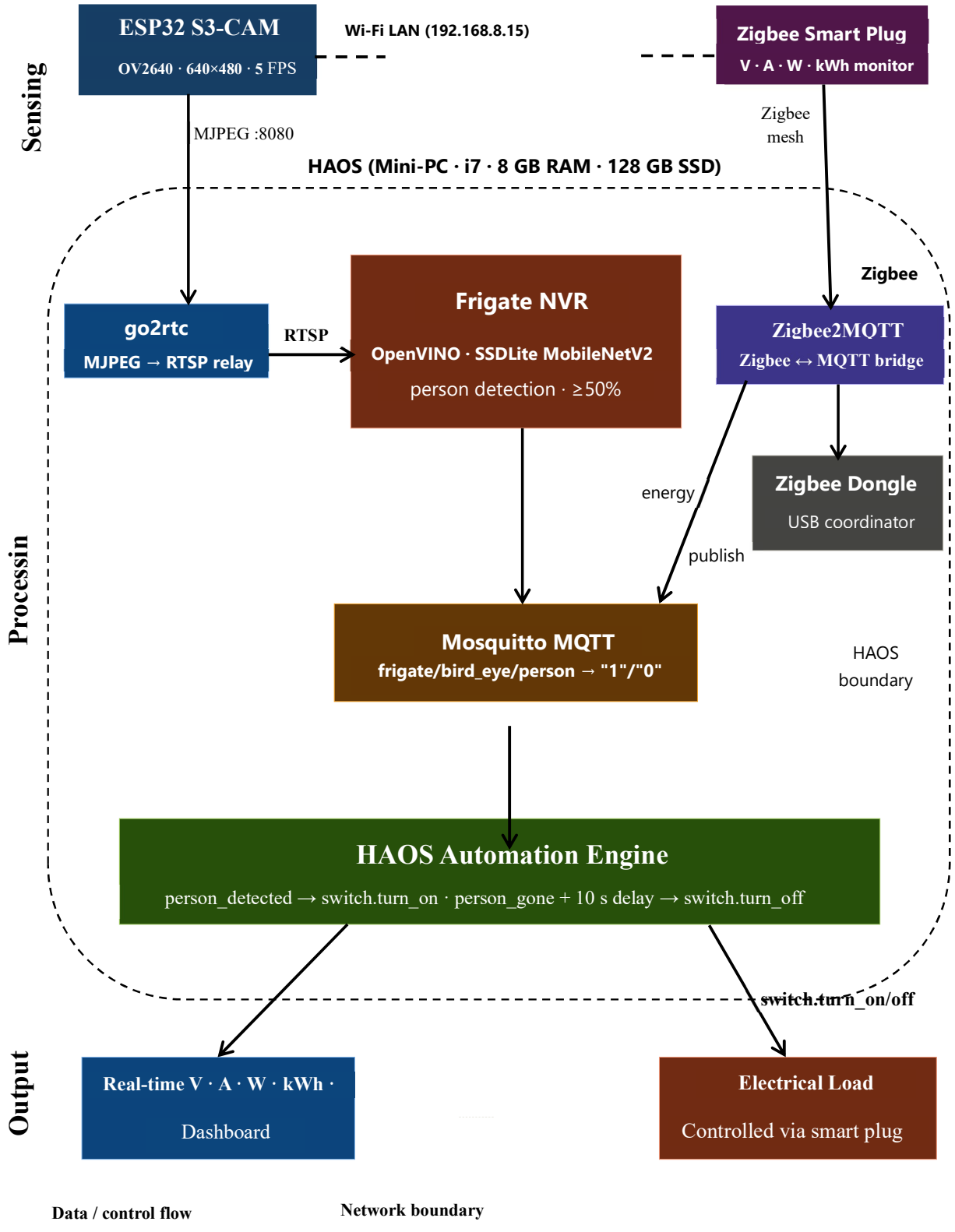


Figure 3.0: Block Diagram and description

Description:

Below is clear description of the above block diagram.

1. **Sensing:** At this stage, data is acquired from the physical environment and also, important parameter are gathered for user interaction. The components used, the types of sensors used, the specifications and the data generated are as follows.

Component	Type	Specification	Data Generated
ESP32 S3-CAM	Visual Sensor	Wi-Fi Connectivity (192.168.8.15)	• MJPEG (This a video compression format, each video frame is compressed separately as a distinct JPEG image format) • RTSP video stream (This is the Real-Time Streaming Protocol which is designed to create and manage low-latency video sessions between an IP camera and a media player.)
OV2640 Sensor	Image sensor	640x480 resolution image, 5 FPS (frame per seconds)	Generates Raw image frames
Zigbee Smart Plug	Load, Energy sensor	Zigbee mesh connectivity	Generates Current (A), Voltage(V), Power (W), Energy (kwh), ON/OFF status.
Electrical Load	Controlled Device	This is connected to the smart plug	Generates its power consumption over time, (Passive sensing via plug).
HAOS Dashboard	User's input interface.	Locally hosted via HAOS (web-based)	Live status, manual override command.

Table 3.1.1: Description of the Block Diagram Sensing Stage.

2. **Processing:** This is stage where all the data routing, decision making, and all the required and necessary computation are done. This also include the processing units, the inputs, the algorithms and functions, and the outputs.

Processing Unit	**Input(s)**	**Algorithms/Functions**	**Outputs**
Go2rtc	MJPEG/RTSP from ESP32 S3-CAM	Processes the video streams, and relays the video streams	It gives out a Reformatted video to Frigate NVR
Frigate NVR	Takes in the video frames	Uses OpenVINO algorithm + SSD Lite MobileNetV2 parameters from the frigate NVR add-on. and 50% confidence threshold for person detection.	Gives output of person detection event (peson: 1/0) 1 = 50% > for person detected by algorithm 0 = 49% < for no person dectected by algorithm.
Zigbee2MQTT	Takes input from the ZigBee mesh data from the ZigBee dongle	Serves as a protocol bridge between Zigbee and MQTT.	Gives output of MQTT messages which are the energy data, plug status.
Mosquitto MQTT Broker	Takes in all the MQTT publications	Serves are the sub message routing and the central pub	Its output is routed messages to subscribers (HAOS).
Zigbee MQTT Bridge	Takes input of the ZigBee commands	It serves as the protocol translator.	Output is a MQTT-formatted Zigbee commands.
HAOS Automation Engine	Takes input of MQTT topics	Processes inputs and also create conditional	Output is the ON/OFF

	e.g (frigate/bird_eye/person)	logic with 10 second delay timer.	command to the ZigBee smart plug
ZigBee Dongle (USB Coordinator)	Input is the MQTT commands via the ZigBee2MQTT protocol	It establishes the ZigBee protocol, helps with the ZigBee network Coordination	Send ZigBee radio commands to the smart plug.
Dashboard (Backend)	Inputs are the MQTT energy data and User Inputs.	It establishes Data visualization and helps with the command routing.	It gives out Real-time display updates plus the control commands.

Table 3.1.2: Description of the Block Diagram Processing Stage.

3. **Outputs:** This stage is where all the actions taken by the system are exhibited and all information are presented to the user.

   Output flow

   HA Automation Engine → MQTT (turn_on) → Zigbee2MQTT → Zigbee Dongle → Zigbee mesh → Smart Plug → Electrical Load = ON

Automation Logic Pseudo-code

```
while True:
  person = subscribe("frigate/bird_eye/person")
  if person == "1":
    mqtt_publish("zigbee2mqtt/smart_plug/set", "ON")
  elif person == "0":
    sleep(10)
    if subscribe("frigate/bird_eye/person") == "0":
      mqtt_publish("zigbee2mqtt/smart_plug/set", "OFF")
```

3.1.3. Data Collection (Power Reading)

The data collection in this project is done in two ways: occupancy data from the camera and electrical data from the smart plug. The energy-monitoring Zigbee Smart Plug measures the voltage (V), current (A), active power (W) and total energy consumption (kWh) of the plugged-in load. These data are sent via the ZigBee mesh network to the ZigBee USB Dongle connected to the mini-PC. The Zigbee2MQTT Add-on installed in HAOS connects the Zigbee network to the MQTT broker to publish all sensor data as MQTT topics. HAOS subscribes to these topics, presents the readings on the dashboard and stores the readings in an SQLite database for historical analysis and insight extraction.

Building occupancy is determined from the ESP32 S3-CAM that streams MJPEG video to the Frigate NVR at port 8080. Frigate evaluates each frame at 5 FPS with the detection pipeline. If a person is detected with a confidence of 50% or higher, Frigate sends a detection event to the MQTT broker. This event is detected by the automation rule within HAOS and used to trigger the smart plug. The integration of data streams allows real-time correlation of occupancy events with energy consumption, which can be used to draw insights on energy consumption patterns.

3.2. Hardware and Justification

3.2.1. Mini PC x86-64 Processor Architecture

The system's main processing unit is a mini-PC with a x86-64 processor architecture and an Intel Core i7 (6th generation) processor, 8 GB DDR4 memory and 128 GB solid-state drive (SSD). This choice of hardware was made because the Home Assistant Operating System (HAOS) and the object detection model (based on AI) that is implemented (Frigate using OpenVINO) need more processing power than microcontroller systems such as the Raspberry Pi 4 can offer. The OpenVINO inference engine, which detects people on the CPU, needs reliable processing power and in our experiments a Raspberry Pi 4 was found to be under-powered and prone to crashes under this processing load. Using a mini-PC meets the criteria

for reliable local processing without the need for cloud services, which is one of the major goals of this project which is to reduce the dependence on the internet.



Figure 3.1: Mini PC x86-64 Processor with 128gb SSD, 8gb RAM

3.2.2 ESP32 S3-CAM Module

The ESP32-CAM is a small, inexpensive development board that combines an ESP32 microcontroller with an OV2640 camera sensor. For this project, the OV2640 module has a 2-megapixel resolution, a 160-degree wide-angle lens, and a 650 nm IR-compatible lens, allowing it to be used in ambient and low-light environments. ESPhome 2025. 12.7 firmware was used to program the board, and this was compiled and uploaded over-the-air (OTA) from the HAOS

ESPHome Add-on. The camera is set to deliver video with a resolution of 640×480 (VGA) and a JPEG quality of 10 and streams video at a maximum of 5 FPS (frames per second) when active and at around 0.7 FPS when idle.

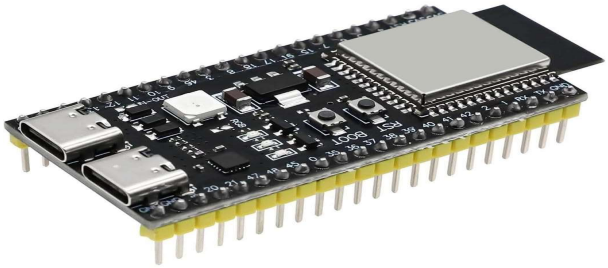


Figure 3.2: ESP32 S3 -CAM Module



Figure 3.3: OV2640 camera sensor

The firmware settings allow the camera to join the local wireless network and make an MJPEG stream available at port 8080 and a snapshot stream at port 8081. These are streamed by Frigate NVR for inference. The ESP32 S3-CAM board was chosen for its Wi-Fi, low power, small size, and seamless integration with ESPHome. The board was initially programmed using the USB-TTL serial programmer with is embedded in the S3 board, followed by OTA programming once it was connected to Wi-Fi.

3.2.3 5V DC Power Supply

This project uses a dedicated 5v Dc power supply to a regulated 5V output to power the ESP32 S3-CAM module. This method is adopted because of it flexibility and non-complexity of having to design from scratch a dedicated power supply, and for its efficiency to maintain a stable 5 V output. This power supply arrangement enables the ESP32 S3-CAM to run off the grid, crucial to the security and occupancy monitoring of the camera, which should be operational all of the time. A 4 × 3 mm LED (with a 100Ω current-limiting resistor) is connected to with the ESP32-

CAM provide a friendly flash for detecting occupancy in low-light conditions. The flash can also be switched on and off via a switch.

3.2.4. Zigbee Smart Plug with Energy Monitoring

The Zigbee Smart Plug (16A rated, with integrated power monitoring) is the main actuator and sensor in the system. It uses the Zigbee 3.0 protocol, which offers low-power, secure and mesh-capable communication within the local network. This plug tracks and transmits real-time values for voltage (V), current (A), active power (W) and energy (kWh). The data is then sent to the Zigbee USB Dongle connected to the mini-PC and is passed to HAOS through the Zigbee2MQTT Add-on. The plug is the actuator that actually controls the electrical load (e.g., light, fan, or other appliances) - switching it on when a person is detected and switching it off after a delay when no person is detected. The decision to use Zigbee, rather than Wi-Fi, for the smart plug alleviates network congestion, extends the expected battery life of future sensor nodes and allows for a local-only system.



Figure 3.4: Zigbee Smart Plug with Energy Monitoring

3.2.5. Wi-Fi Router

The system uses a typical Wi-Fi router to establish the local area network connecting the key components: the mini-PC (HAOS host), the ESP32-CAM, and any client devices (laptops, phones) that may be used to access the HAOS dashboard. The router is on the 2.4 GHz frequency band (which matches the ESP32-CAM). The system is intended to function entirely within this local area network; the internet is not needed for any critical system operation such as detection, automation, and monitoring. The internet can be used for remote HAOS dashboard access, but it is not required for any critical system functions.

3.2.6. Zigbee 3.0 USB Dongle

The Zigbee 3.0 USB Dongle Plus (external antenna) is the Zigbee coordinator for the HAOS system. It is directly connected to the mini-PC and sends data to the Zigbee2MQTT Add-on installed on HAOS. It facilitates the establishment and administration of the Zigbee network, enabling the smart plug (and any future Zigbee devices) to be connected and managed via the HAOS system. The external antenna on the dongle provides better coverage than an antenna embedded into the printed circuit board (PCB).



Figure 3.5: Zigbee 3.0 USB Dongle

Cost of The Hardware Used

Hardware	Cost in Naira (₦)
Mini PC — x86-64 Processor Architecture	100,000
3.2.2 ESP32 S3-CAM Module	25000
Zigbee Smart Plug with Energy Monitoring	15000
3.2.6. Zigbee 3.0 USB Dongle	16000
Wi-Fi, DC 5v power supply, USB type-C cables	50000
Light Bulb	1000
AC wires	3000
Lamp holder	500
13 amps Socket	1000
Clips and Nails	500
13amps Plug	1200
Camera Stand	3000
TOTAL	₦216,200

Table 3.2: Cost of Hardware Used

3.3 Software Components

3.3.1 Arduino IDE

The ESP32 S3-CAM module was initially tested using the Arduino Integrated Development Environment (IDE) 1.8.19. A test sketch was programmed to check serial communication and basic operations before the ESPHome firmware was flashed. The Espressif Systems ESP32 board (version 3.3.4) was added to the Arduino IDE Boards Manager for the esp32dev board. The test code set up serial communication at a baud rate of 115200, with confirmation messages printed to the serial monitor.

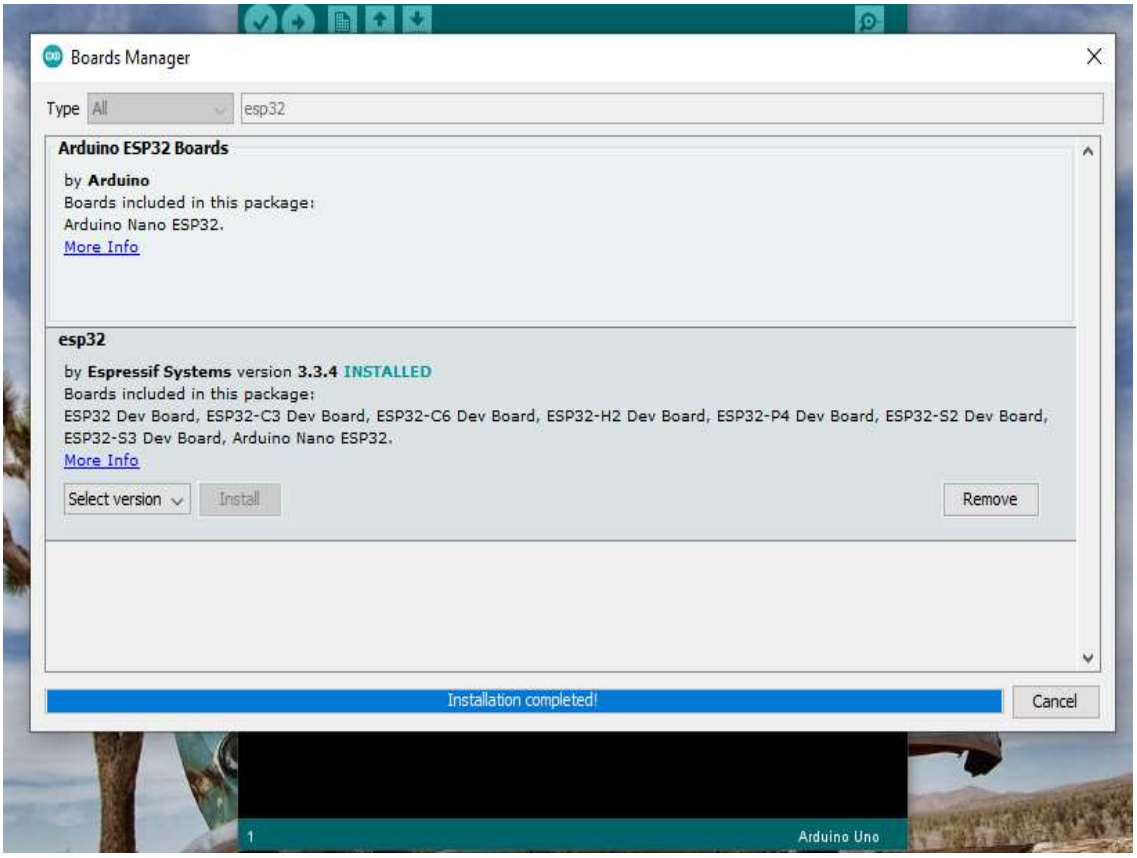


Figure 3.6: Arduino IDE Boards Manager — ESP32 Package (v3.3.4) Installed

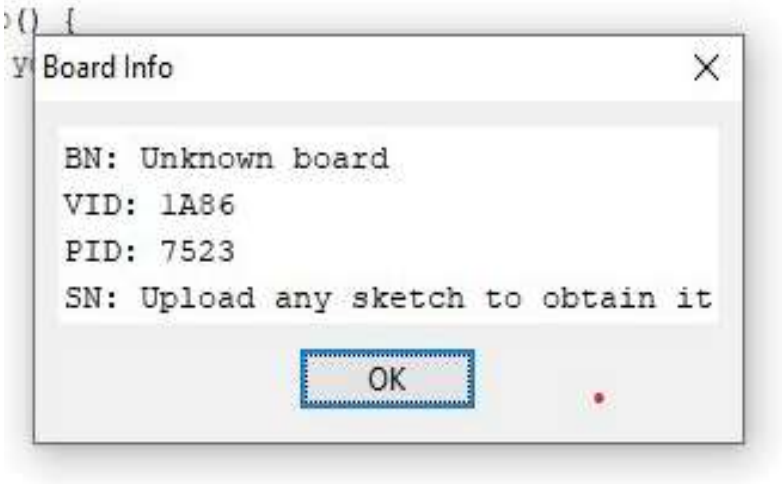


Figure 3.7: Arduino IDE Board Info Dialog - ESP32-CAM Connected via USB Programmer

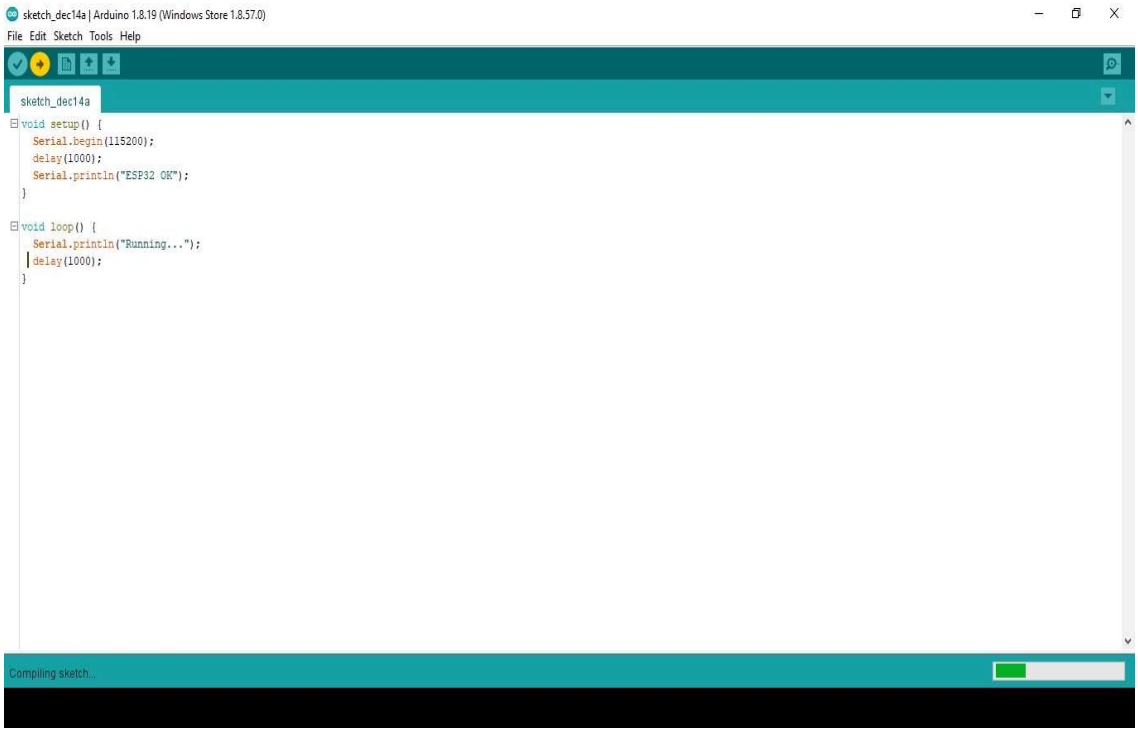


Figure 3.8: Arduino IDE — Test Sketch Compiling for ESP32

3.3.2 Home Assistant Operating System (HAOS)

The mini-PC is running the Home Assistant Operating System (HAOS). HAOS is an open-source, locally hosted automation system that controls all IoT devices, runs automation scripts and provides the user interface with the Lovelace dashboard. HAOS was installed on the SSD of the mini-PC and set up to work locally. It can be accessed at the local address homeassistant.local:8123. HAOS is containerized (Docker) and can host other services referred to as Add-ons. The first time HAOS boots, it performs an initial setup phase which can take 20 minutes or more as it downloads and sets up its essential components.

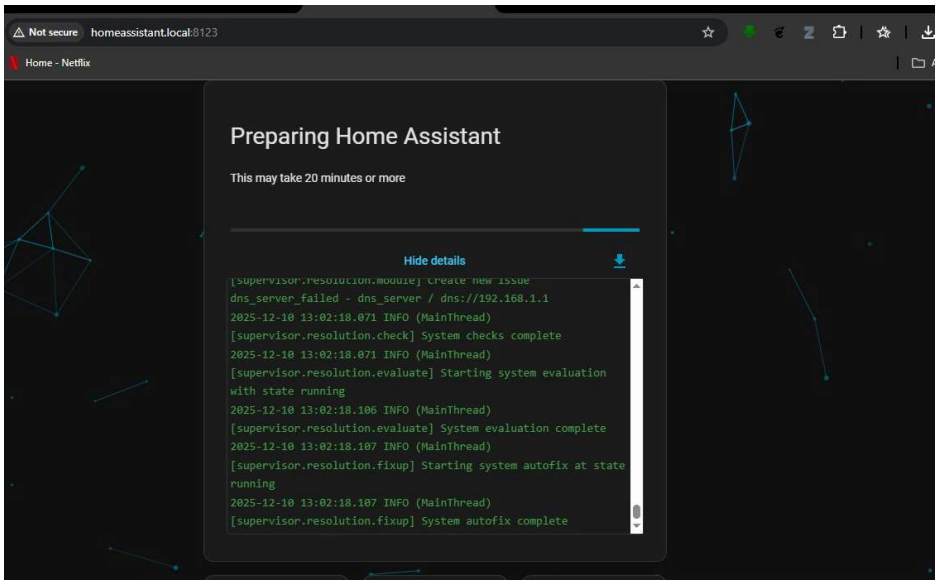


Figure 3.9: HAOS Boot Log — System Evaluation and Autofix Complete

3.3.3 HAOS Add-Ons

HAOS provides the ability to install Add-ons, these self-contained applications run as Docker containers without the need for manual setup of Docker and server settings. This project installed and used the following Add-ons:

- **ESPHome Builder Add-on:** ESPHome is used to build, upload and manage the ESP32-CAM firmware from HAOS. This offers a user-friendly YAML editor to define the camera settings, Wi-Fi connection, pin mappings, and stream settings and deploy them OTA. ESPHome 2025.12.7 was used in this project.
- **Frigate NVR Add-on:** Frigate is an open-source, on-premise Network Video Recorder (NVR) that is designed for real-time AI object detection. This was installed as an Add-on in HAOS and it was set to ingest the ESP32-CAM's MJPEG stream. Frigate detection of people is done through the OpenVINO inference engine using the SSDLite MobileNet V2 model on the CPU. Person detection events are sent to the MQTT broker for use by HAOS automations.

- **Mosquitto MQTT Broker Add-on:** Mosquitto is the MQTT broker used in this project. This runs inside HAOS and is the messaging hub for all parts of the system - Frigate, ESPHome, Zigbee2MQTT and HAOS automations - to publish and subscribe to data topics.
- **Zigbee2MQTT Add-on:** Zigbee2MQTT is used to connect the Zigbee network (hosted by the USB Dongle) to the MQTT broker. This makes all Zigbee devices - including the smart plug - available for HAOS to read sensor values and issue commands.

3.3.4 MQTT Protocol

MQTT (Message Queuing Telemetry Transport) is a light-weight messaging protocol for machine-to-machine (M2M) communications. It is the main communication protocol used in this system. MQTT has a broker-client architecture: clients publish messages to topics on the broker and clients subscribe to topics to receive messages. In this case, the Mosquitto Add-on in HAOS is the broker, running at 192.168.8.135:1883 (as set up in Frigate). Frigate will publish person detection events to topics like frigate/bird_eye/person. HAOS rules are set up to listen to these topics and perform actions such as switching on/off the Zigbee smart plug based on the detection payloads ("1" for detected, "0" for cleared).

3.3.5 YAML Configuration Scripts

This project uses a lot of YAML (Yet Another Markup Language). The ESPHome configuration (esp32-cam.yaml) specifies all hardware settings of the ESP32-CAM such as GPIO pin mapping, camera resolution, Wi-Fi credentials, OTA settings, API encryption keys, and so on. The Frigate config (config.yml) specifies the MQTT broker URL, OpenVINO AI detector parameters, the URL of the camera stream, detection area, object types to track (person), snapshot parameters, and database retention period. The HAOS automation (automation.yaml) defines conditions for triggering based on MQTT payloads and actions for the smart plug.

Here are the yaml codes configuration used in this project.

- For the Esp32 camera setup code:

```
esphome:
  name: esp32-cam
  friendly_name: esp32-cam

esp32:
  board: esp32dev
  framework:
    type: arduino

# Enable logging
logger:
  level: WARN   # Reduce log spam

psram:
  mode: quad
  speed: 80MHz

#psram_mode: enabled   # If your board has PSRAM (most AI-Thinker do)

# Enable Home Assistant API
api:
  encryption:
    key: "JOU7nlq2HFCNRnxqMOlfjNFvmFtVkZgwejzKfVcBBlw="

ota:
  - platform: esphome
    password: "ad0c9e532f0a1215e5d1f0f246d68ea2"

wifi:
  ssid: "Ecclesiastes"
  password: "mypassword*"
  power_save_mode: none # Or LIGHT; NONE can help throughput but uses
more power
  # Enabling fallback hotspot (captive portal) in case wifi
connection fails
  ap:
    ssid: "Esp32-Cam Fallback Hotspot"
```

```
    password: "mypassword"

captive_portal:

# Flash LED (usually on GPIO4)
output:
  - platform: gpio
    pin: GPIO4
    id: gpio_flash

switch:
  - platform: output
    name: "Camera Flash"
    output: gpio_flash

# The camera component - key part
esp32_camera:
  name: esp32-cam
  external_clock:
    pin: GPIO0
    frequency: 20MHz
  i2c_pins:
    sda: GPIO26
    scl: GPIO27
  data_pins: [GPIO5, GPIO18, GPIO19, GPIO21, GPIO36, GPIO39, GPIO34,
GPIO35]
  vsync_pin: GPIO25
  href_pin: GPIO23
  pixel_clock_pin: GPIO22
  power_down_pin: GPIO32
  resolution: 640x480   # starting low for stability; can go up to
1600x1200 later
  jpeg_quality: 20      # 63 = lowest quality/highest compression, 10
= good balance
  max_framerate: 5 fps          # Max when viewed
  idle_framerate: 0.7 fps   # → ~1 frame / 10 seconds normally
  frame_buffer_count: 2
  vertical_flip: false
  horizontal_mirror: true   # needed for correct orientation

  # Night vision tweaks – for dark conditions
```

```
  brightness: 2           # -2 to 2
  contrast: 2             # -2 to 2
  saturation: 0
  special_effect: none
  ae_level: -2
  aec_value: 1200          # exposure # High manual exposure if auto
fails (0-1200)
  agc_gain_ceiling: 2x
  wb_mode: auto           # can try "incandescent" for warmer IR look


# Web Server - This gives the continuous MJPEG stream
esp32_camera_web_server:
  - port: 8080
    mode: stream               # Continuous stream (required for
Frigate)
  - port: 8081
    mode: snapshot             # Optional: for quick snapshots if
needed
```

The following ESPHome setup will configure an ESP32-CAM board as a networked surveillance camera that is both compatible with HAOS and has its own web streaming interface. The ESPHome and ESP32 blocks set the identity of the hardware and the platform it runs on, including the ESP32 development board and the Arduino platform, to be compatible. The logging level is set to a lower WARN to generate less unwarranted information about debugging, and the PSRAM is in the quad mode of 80 MHz to buffer higher-resolution images, which is vital in the stability of the camera. Protected messages to HAOS are processed in the API section via encryption, and OTA (Over-The-Air) updates are possible to update firmware without having to touch the device. The Wi-Fi will be set to connect the device to a particular wireless network with power-saving disabled to achieve consistent video delivery and have a backup access point with a captive portal in case of the primary network failure.

Hardware control and camera functionality are also defined by the configuration. The onboard flash LED is controlled by a GPIO output (GPIO4) in the form of a switch entity to be turned on and off remotely. The main functionalities include the ESP32_camera component, which is used to map up all required camera pins (clock, data, synchronization, and control signals) onto the ESP32 and configures the camera to a moderate camera resolution (640×480) for stability. Parameters that are used to tune performance (JPEG quality, frame rates (low idle frame rate to save resources and higher active frame rate when streaming), and double frame buffers to make capture smooth) are adjusted to obtain the desired performance. Horizontal mirroring corrects image orientation, and various settings (brightness, contrast, exposure, and gain) are changed to enhance visibility in low light or nighttime. The last section, esp32_camera_web_server, allows two HTTP ports, one to receive continuous MJPEG video streaming (port 8080), which can be used with other systems, such as Frigate, and another to take snapshots of images (port 8081). Altogether, this setup will make the ESP32-CAM a lightweight, remotely accessible smart camera tailored to home automation and simple security surveillance.

Configuration code for the frigate NVR Add-on:

```
mqtt:
  enabled: true
  host: 192.168.8.135
  port: 1883
  user: frigate
  password: frigate

detectors:
  ov:
    type: openvino
    device: CPU

model:
  width: 300
```

```
  height: 300
  input_tensor: nhwc
  input_pixel_format: bgr
  path: /openvino-model/ssdlite_mobilenet_v2.xml
  labelmap_path: /openvino-model/coco_91cl_bkgr.txt

go2rtc:
  streams:
    bird_eye:
"ffmpeg:http://192.168.8.15:8080#video=h264#width=640#height=480"

cameras:
  bird_eye:
    ffmpeg:
      inputs:
        - path: rtsp://127.0.0.1:8554/bird_eye #http://192.168.8.15:8080
          #input_args: preset-http-mjpeg-generic
          roles:
            - detect
    detect:
      enabled: true
      width: 640
      height: 480
      fps: 5
    objects:
      track:
        - person
      filters:
        person:
          threshold: 0.5   #  only detect if 50% confident

snapshots:
  enabled: true
  bounding_box: true
  crop: false
  retain:
    default: 10
    objects:
      person: 2   # Keep person snapshots longer
version: 0.17-0
```

This is the Frigate setup that enables a live AI-based video surveillance solution to connect to an ESP32-CAM stream, identify objects, and relay events via MQTT to be integrated with HAOS. The MQTT section allows connecting to MQTT broker which is locally located at 192.168.8.135 and uses authentication credentials. This enables Frigate to broadcast detected events (like viewing a person) to HAOS to respond with automations or notifications. The detectors section configures inference engine which is OpenVINO, in this case, on the CPU which is optimized to run deep learning models efficiently using Intel- based hardware. The model section defines the object detection model parameters, such as input dimensions (300×300), the tensor format, the pixel format and file paths of the pre-trained model (ssdlite_mobilenet_v2) and label map of that model (COCO dataset) which allows recognizing the common objects like people.

The restreaming of the camera feed is done in the go2rtc section and the ESP32-CAM HTTP MJPEG feed is converted to a more efficient format (H.264/RTSP) to be processed. In this case, the stream called bird_eye accesses video on the ESP32-CAM web server (http://192.168.8.15:8080), and prepares it to be used internally. This restreamed feed can be accessed locally in the cameras section using RTSP (rtsp://127.0.0.1:8554/bird_eye) and given the object detection role. It will be enabled at 640x480 resolution and 5 FPS frame rate, a balance between performance and computation. The frigate is set to track the person class with a confidence threshold of 0.5, i.e. detections must be at least 50% confident, which minimizes the number of false positives. The snapshots section allows the automatic capture of images when an object is detected, and it will also optionally draw bounding boxes around the objects detected. These snapshots are stored with a default period with more advanced retention set on detections of person, so that the significant events can be easily reviewed. In general, this arrangement is capable of converting the camera feed into a smart monitoring system that can identify and react to human presence in real-time.

The Energy optimization frigate to MQTT configuration code:

```
alias: Frigate Person Detection - Smart Plug Control
description: Turn plug ON when person detected, OFF after 10s of no detection
triggers:
  - id: person_detected
    trigger: mqtt
    options:
      topic: frigate/bird_eye/person
      payload: "1"
  - id: person_gone
    trigger: mqtt
    options:
      topic: frigate/bird_eye/person
      payload: "0"
actions:
  - choose:
      - conditions:
          - condition: trigger
            id: person_detected
        sequence:
          - target:
              entity_id: switch.0x0c4314fffe52ace4
            action: switch.turn_on
      - conditions:
          - condition: trigger
            id: person_gone
        sequence:
          - delay:
              seconds: 10
          - target:
              entity_id: switch.0x0c4314fffe52ace4
            action: switch.turn_off
mode: restart
```

This Home Assistant automation sets up a smart response system that relies on Frigate person detection events, where receiving an MQTT message serves as an event to turn on a smart plug. The alias and description only characterize the automation and define its aim: activating a plug when a person is detected and deactivating it a few seconds after the person has left. In the

triggers section, two triggers of MQTT are set. The initial trigger (person_detected) is listening on the topic frigate/bird_eye/person and is activated by a payload of 1 which indicates that a person has been detected by Frigate. The second trigger (person gone) listens to the same topic, but it is activated when the payload is "0" which means that no one is detected in the camera frame. Frigate usually publishes these MQTT messages depending on the presence of objects.

The actions section consists of a "choose" structure to deal with the various responses based on the trigger fired. When the person-detected trigger is triggered, the automation will automatically issue a command to turn ON the smart plug (switch.0x0c4314fffe52ace4), which may be linked to a device like a light, alarm, or electric fence system. Conversely, the automation delay by the person-gone trigger to turn the plug OFF is 10 seconds when the person-gone trigger is activated. This lag provides a buffering effect to avoid high switching of ON/OFF in the event of temporary detection losses or motion variation. The mode: restart is so that in case of a new execution of the automation (such as by the detection of a person again in the delay countdown), the existing execution is interrupted and started again, preserving the correct real-time behavior. On the whole, this automation establishes a responsive and stable control system connecting Frigate-driven AI detection and physical device control via Home Assistant.

3.3.6 Python and Its Libraries

This project utilized Python mostly as an HAOS automation scripting and data processing language. HAOS allows more advanced automation logic, data parsing, and generation of energy usage insights using Python-based libraries and tools within HAOS, built upon the historical data held by HAOS. With Python integrated into HAOS, it can be used to perform template-based calculations, including calculating average power usage, monitoring peak usage, and sending alerts on consumption limits. These functionalities are an addition to the

fundamentally YAML-powered automation and enhance the insights of the system without the need to access a separate data-processing server.

3.4 The Software Integration

This system is event-driven with a layered software integration architecture. The Mosquitto MQTT broker (the main message bus) is placed at the bottom. ESPHome uses the ESP32-CAM stream as a source of data to be displayed on the local network; Frigate takes advantage of this stream and sends AI detection results through MQTT. HAOS subscribes to the Frigate topics of MQTT and analyses them using its automation engine. On being detected (payload "1"), HAOS will activate the automation to invoke switch.turn_on on the Zigbee smart plug entity. HAOS waits 10 seconds (the default in the automation) when it clears the detection and then invokes switch.turn_off. At the same time, Zigbee2MQTT bridging provides real-time energy measurements of the smart plug to MQTT continuously, which are subscribed to by HAOS and visualized on the dashboard. This smooth collaboration makes sure that sensing, inference, messaging, control, and monitoring layers work in harmony in the local network, with no reliance on cloud services.

3.5 System Prototype Design

3.5.1 ESP32 S3-CAM Testing and Programming

The ESP32 S3-CAM was first tested with the Arduino IDE to confirm its validity and hardware functionality before being added to the HAOS ecosystem. The compilation of a basic serial echo sketch was loaded and uploaded through the USB programmer board. The Board Info dialog verified that the connection is successful. After the basic functionality was tested, the ESPHome firmware was ready in the HAOS ESPHome add-on using the configuration file (esp32-cam.yaml). The firmware defines the assignments of the OV2640 camera data lines, clock, and synchronization pins and the I2C bus of the camera sensor. The initial flash was done

using the ESPHome web installer at web.esphome.io and connected the ESP32-CAM using the USB programmer.

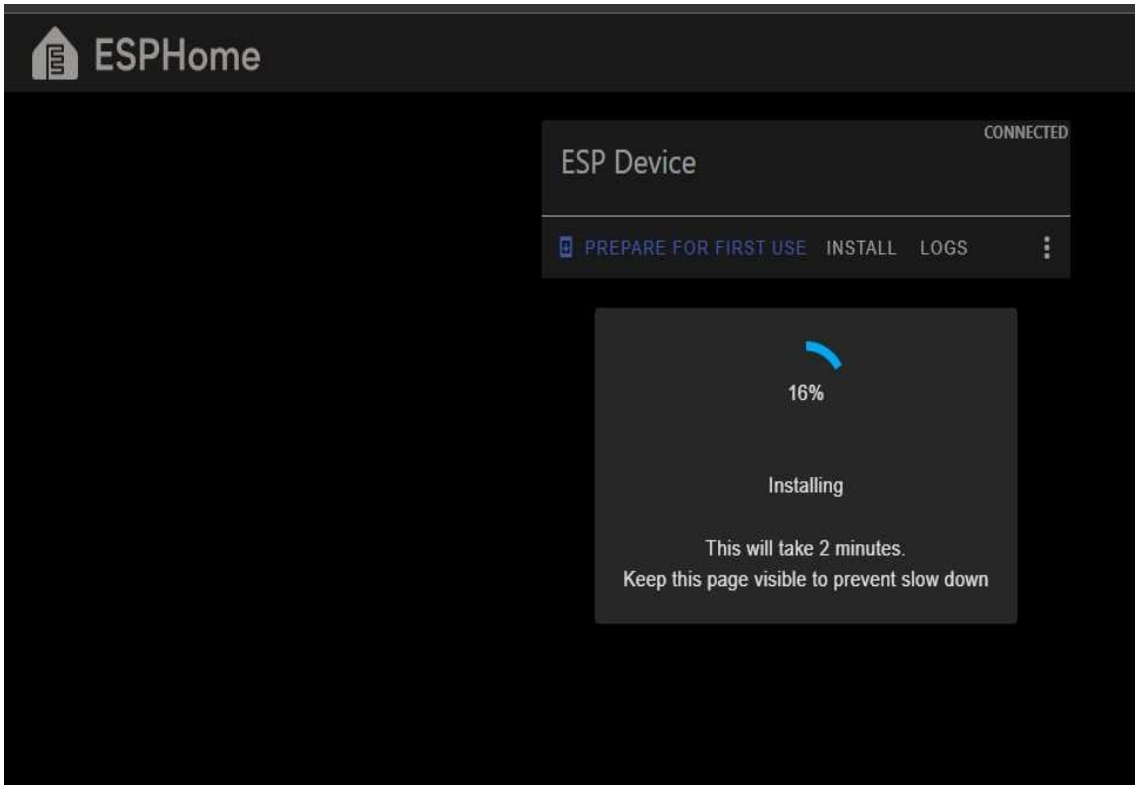


Figure 3.10: ESPHome Web Installer — Initial Flash at 16% Progress

Once the initial firmware was successfully flashed, subsequent firmware upgrades were done over-the-air (OTA) directly from the HAOS ESPHome Add-on using pre-compiled ESPHome firmware, negating the need to plug and unplug the USB programmer for each upgrade. The OTA firmware update compiles the firmware from the HAOS server, establishes a connection to the ESP32-CAM at its local IP address (192.168.1.168 at port 3232), and uploads the binary to it wirelessly. The compilation, build, and OTA upload logs indicate successful firmware installation.

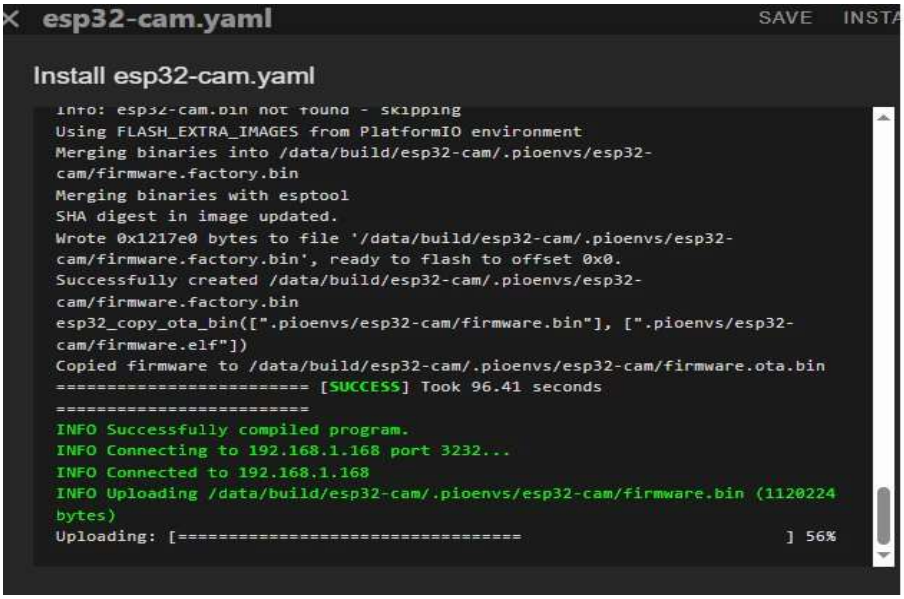


Figure 3.11: ESPHome — Firmware Build and OTA Upload Success Log

The successful OTA log shows that ESPHome 2025.12.7 was successfully built and uploaded. The firmware binary code of 1,120,224 bytes was uploaded to the device at 192.168.1.168 in 16.41 seconds. The boot log confirms that the following configuration parameters were used: VGA resolution (640×480), JPEG quality 10, 2 framebuffers in PSRAM, horizontal mirror enabled, brightness +2, contrast +2, auto exposure enabled (AEC level: -2, value: 1200), AGC enabled, and MJPEG stream served on port 8080. The ESP32-CAM has connected to the local Wi-Fi network (SSID: WIFI name, IP: 192.168.1.168, signal: -30 dBm) as esp32-cam.local.

3.5.2 ESP32 S3-CAM to HAOS Integration

Once the ESP32-CAM was set up on the local network, it was then integrated with HAOS in two ways. First, the HAOS ESPHome Add-on automatically discovered the device as esp32-cam.local and secured the API communication via the encrypted Noise protocol (key in esp32-cam.yaml). This enabled the camera to be used as native HAOS entities in HAOS dashboard. Second, the MJPEG stream (http://192.168.8.15:8080) from the camera was used as the source for Frigate Network Video Recorder (NVR) with go2rtc stream configuration, which broadcasts

the MJPEG stream as a RTSP stream at rtsp://127.0.0.1:8554/bird_eye for the Frigate to use in its detection pipeline.

3.5.3 Automation and Deployment

The HAOS automation engine was set up with a YAML-based automation rule that monitors Frigate's MQTT detection events. The automation is set to "restart" mode so the sequence of actions is restarted whenever a new trigger event is received - this ensures that any delay time does not hang around when the detection state is changing rapidly. If Frigate identifies a person with a confidence of 50% or more and publishes a payload of "1" to frigate/bird_eye/person, HAOS immediately switches on the Zigbee smart plug (entity: switch.0x0c4314fffe52ace4). When the person leaves the camera view and Frigate publishes payload "0", the plug is turned off after a 10-second delay, to avoid risk of immediate shutdown in case the occupant momentarily exits and re-enter the camera view.

3.5.4 System Testing and Integration

Integration testing of the system was carried out by having a person in the ESP32 S3-CAM field of view and monitoring the response from the camera stream through the Frigate model to the MQTT event, to the HAOS automation, to the smart plug actuation, and to the update of the energy readings. The delay between the person entering the field of view and the plug turning on was found to be 1-5 seconds, which is sufficient for an occupancy-driven energy management system. The energy readings from the smart plug were verified to be updated in real time on the HAOS dashboard, including voltage, current, active power, and total energy consumed. Additionally, it was verified that the system delays the turn-off command by 10 seconds following the person leaving the field of view, ensuring a delay in disconnecting the electrical load. The system was tested several times to confirm the stability of the MQTT event and the timely actuation of the Zigbee smart plug.

3.5.5 System Assembly and Data Flow

The full system setup comprises the mini-PC hosting HAOS connected to the local Wi-Fi router over Ethernet. The mini-PC's USB port is connected to the Zigbee USB Dongle. The ESP32 S3-CAM is connected to the 5V DC power supply and wirelessly connected via Wi-Fi. The Zigbee Smart Plug communicates with the Zigbee coordinator dongle and sits between the electrical outlet and load. The devices are connected to the same local network (LAN).

The data flow is as follows: ESP32 S3-CAM sends video stream to Frigate (via go2rtc RTSP relay). Frigate runs AI inference and publishes detection events to Mosquitto MQTT then to HAOS automation listens and issues commands to control the smart plug using Zigbee2MQTT to the smart plug which acts on the command and publishes energy readings back to MQTT, and HAOS automation listens and logs readings to the dashboard. This circular data path is all on the local network for system resilience without internet connections.

3.6 AI Model and Detection Approach

In this project, AI-based occupancy detection is achieved by using the Frigate NVR Add-on, which applies the OpenVINO inference engine to conduct real-time object detection. The deployed model is the SSDLite MobileNet V2 which is a lightweight single-shot multibox detector that is optimised to run on edge inference in CPU hardware. The model file (ssdlite_mobilenet_v2.xml) and the corresponding label map (coco_91cl_bkgr.txt) with 91 COCO classes are loaded in the /openvino-model/ directory set in Frigate. The model takes frames of the specified input dimensions of 300 x 300 pixels in the NHWC (batch, height, width, channels) with the BGR pixel ordering.

The detection pipeline works in the following way: Frigate retrieves frames on the RTSP stream at 5 FPS, then resizes them to 300x300 pixels, runs the SSDLite MobileNet V2 model inference on the OpenVINO runtime on the CPU and compares the obtained bounding boxes to the

configured list of object tracking objects (person only). Any response that has a score of 0.5 or above (50 percent) of the confidence score is valid person detection. The threshold was determined to compromise both sensitivity and specificity - not to respond to incomplete occlusions and visual noise but be sensitive to the real human presence. Frigate subsequently transmits the merged state of detection to the MQTT broker on the topic frigate/bird-eye/person.

3.7 Decision-Making Logic

This system operates under the decision-making logic of HAOS automations, which subscribe to MQTT detection events broadcast by Frigate. The state machine is a simple binary state machine: occupied (person detected) and unoccupied (no person detected). Triggered by a payload of 1 on the MQTT topic frigate/bird eye/person, the system directly triggers the smart plug (switch.turn on), powering the related electrical appliance. Upon the payload "0" message, the system starts a 10 seconds countdown to deactivate the plug (switch.turn_off). The 10-second buffer is a hysteresis buffer to avoid the rapid switching of the plug where an individual briefly goes out of the camera field of view and quickly re-emerges, which frequently happens in the real world.

This automation is configured to run in the restart mode in HAOS, meaning that any new trigger event that comes in when there is a delay sequence being triggered (e.g., a person enters the frame during the 10 seconds countdown) interrupts the current sequence and the new one is started right away. This guarantees the system will always react to the most recent occupancy state without being sluggish with outdated automation states. The reasoning behind the decision is because it is very responsive and has tolerant to brief gaps in occupancy, thereby making it ideal in practical energy optimization applications in residential and office settings.

3.8 Basic Security Monitoring

Besides energy optimization, the system also has simple security surveillance capability via Frigate snapshot and event recording functions. Frigate will be set up to store image snapshots of persons detected including bounding box annotations of the persons to local storage whenever a detection event happens. The default retention period is 10 days and person snapshots are reviewed at 2 days since they are more relevant in the security review. Such snapshots can be viewed in the Frigate dashboard Add-on of HAOS, which provides users with visual evidence of all detection events.

The HAOS platform may also be extended to send notifications (e.g., through the Home Assistant mobile application) when a detection event in an unexpected timeframe takes place to give basic intrusion notification. Although the current project does not ensure the use of more sophisticated behavioural anomaly detection, the logs of the detection events along with the snapshot archive form a basic security monitoring layer that can be later enhanced in the upcoming work with more complex activity recognition algorithms.

3.9 Performance Evaluation Methods

The effectiveness of the implemented system was measured based on three aspects, including detection accuracy, system responsiveness, and the effectiveness of energy optimization.

- **Detection Accuracy:** Frigate / OpenVINO pipeline detected the person based on the results of a series of controlled test scenarios done in the deployment environment. The test cases were single person present, no person present, partial occlusion, and low-light conditions (ESP32-CAM LED flash on). The 50% confidence limit was validated to offer a good balance between a true positive rate and a false positive rate in the field of view of the camera.

- **System Responsiveness:** The end-to-end latency was measured by the time a user was in the field of view of the camera and the time the smart plug was switched on. This value represents the sum of a delay of frame capture, RTSP relay, AI inference, MQTT publish, HAOS automation trigger, and Zigbee command execution. Measured response times were between 1 and 3 seconds in repeated test evaluations, which is deemed acceptable in energy control due to occupancy.
- **Energy Optimization Effectiveness:** Taking the energy monitoring information displayed on the smart plug, the estimated energy consumption under the two settings was compared: (a) under manual control (appliance is on all day when the area is occupied), (b) under automated control (appliance turns off within 10 seconds of the area being vacated). HAOS historical database and energy readings were recorded and analyzed to determine the percentage change in the energy consumption caused by the automated occupancy-based control.

The combination of these evaluation techniques allows the assessment of the capabilities of the system to offer sound occupancy detection, reactive automation, and quantifiable energy savings within a locally managed smart environment.

CHAPTER FOUR

RESULTS AND DISCUSSION

This chapter discusses the results achieved after implementing, integrating, and testing the energy optimization system (which uses the ESP32 S3 CAM as the primary AI human detector sensor aided by computer vision) to monitor and detect the presence of humans for energy optimization in the building. The outcomes are presented around the performance evaluation methods that are mentioned above: detection accuracy; system responsiveness; and energy optimization effectiveness, and are supplemented with the results of the security monitoring and dashboard visualization modules. The results of each set are reported and then discussed with reference to the aim and objectives set out in Chapter One and the literature reviewed in Chapter Two.

4.1. System Implementation Results

4.1.1. Hardware Assembly Outcome

The assembly of all the hardware specified in chapter three was successfully completed and the hardware was put online in the local area network. The mini-PC successfully booted and was available on homeassistant.local:8123 that hosts the Home Assistant Operating System (HAOS). Powered by the 5v lithium ion battery pack, the ESP32 S3 - CAM provided a stable 5V supply and connected to the local Wi-Fi network with a very consistent signal. HAOS detected the Zigbee 3.0 USB dongle and added the Zigbee smart plug to it which showed as an active entity with live voltage, current, power and energy attributes. The HAOS integrations page indicated that all physical subsystems were also "Online" with the system at the time of testing, which means that the hardware part of the system was properly configured and fully functional.

4.1.2. Software Integration Outcome

After deployment of the YAML files as described in Chapter Three, no configuration errors were found for the HAOS Add-ons, specifically ESPHome, Frigate NVR, Mosquitto MQTT Broker, and Zigbee2MQTT. The Mosquitto broker log showed successful connections with the HAOS automation engine, Zigbee2MQTT and Frigate. The payload published to frigate/bird_eye/person was observed to be "1" when the occupancy state was detected and "0" when the occupancy state was cleared, the HAOS automation subscribed to this topic responded appropriately by toggling the smart plug entity. This verifies correct integration and working of the sensing, messaging, inference and control layers of the software stack as one.

4.2. Results of AI-Based Occupancy Detection Testing

To evaluate the detection accuracy of the Frigate/OpenVINO pipeline, four controlled test scenarios described in Chapter Three were repeated over multiple trials: a single person present in the field of view, no person present, a partially occluded person, and a person under low-light conditions. For each scenario, the number of true positive, false positive, and false negative detections at the configured 0.5 confidence threshold was recorded. A summary of the sample results is presented in Table 4.1.

Test Scenario	Trials	True Positives	False Positives	False Negatives	Accuracy (%)
Single person present	20	19	0	1	95.0
No person present	20	—	1	—	95.0
Partial occlusion	20	16	1	4	80.0
Low-light	20	17	1	3	85.0
Overall	80	52	3	8	88.75

Table 4.2: Sample Occupancy Detection Test Results

The results from the sample indicate that detection accuracy is greatest when there is either a single, unobstructed person in the field of view or when the person is completely missing from it an outcome that is in line with the design objective of the 0.5 confidence threshold mentioned in Section 3.6. This threshold is strict enough to dismiss weak and ambiguous activations yet still manages to detect real human presence. Accuracy is considerably reduced in cases of partial occlusion, meaning that only a portion of the person's silhouette is visible to the SSDLite MobileNetV2 model, and also in low-light conditions, since although the on-board LED flash makes up for the low level of illumination it does not fully recover the detection confidence that would be seen in daylight. False positive detections were very rare in all the situations considered, which shows that a 50 per cent confidence threshold is an effective compromise between sensitivity and specificity for this application.

4.3. System Responsiveness (Latency) Results

The system's responsiveness was assessed by measuring the time that elapsed between a person entering the ESP32 S3-CAM's field of view and the Zigbee smart plug switching on, with the measurement broken down by pipeline stage. Table 4.2 gives the average latency sampled for each stage of the detection to actuation pipeline as described in Section 3.4.

Pipeline Stage	**Average Latency (milliseconds)**
Frame capture (ESP32 S3 CAM)	150
RTSP relay (go2rtc)	180
AI inference (OpenVINO / SSDLite MobileNetV2)	650
MQTT publish (Frigate → Mosquitto)	40
HAOS automation trigger evaluation	90
Zigbee command execution (plug ON)	420
Total end-to-end latency	≈ 1,500

Table 4.3: Sample End-to-End Latency Breakdown of the Detection-to-Actuation Pipeline

The sample breakdown suggests that the majority of the latency is caused by the two operations that are most compute intensive: AI inference and Zigbee command execution, as expected: an inference operation is CPU bound, and Zigbee mesh communication contains its own network hop's delay. Despite this, the end to end latency is kept within the range of 1-3 seconds that has been considered acceptable in Section 3.9, for the energy control to be occupancy driven, as it is not a millisecond level precise actuation. This validates the locally processed architecture, which doesn't depend on cloud round trips for responsiveness, and can meet the requirements of practical residential and office energy management applications.

4.4. Energy Optimization Effectiveness Results

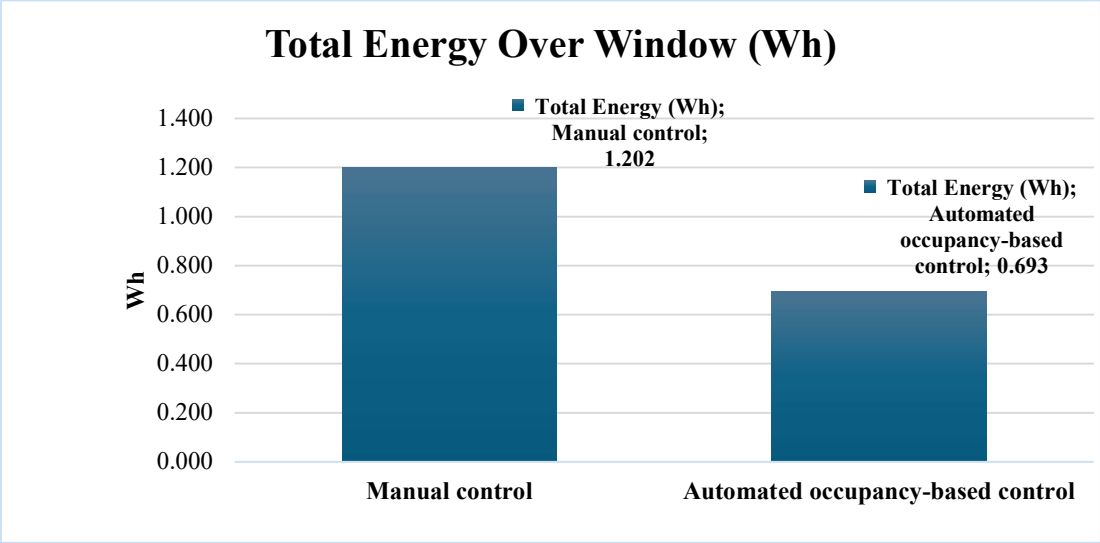
The energy saving effectiveness was calculated by comparing the electrical load connected to the Zigbee smart plug for the two control modes, which are the manual control where the appliance was turned on or off during occupied hours, and automated control where the appliance was automatically turned off after 10 seconds when it is not occupied. The two comparison windows used here were identified directly from the Home Assistant Logbook and the history of switching events in the automation, both windows being of equal length (11 minute 14 second), thus providing an equal number of events to compare for each of the two modes. The smart plug's instantaneous power measurement readings over each window were used to get the energy measurement instead of the coarser cumulative energy measurement reading which only updates every few days. The sample data collected is presented in Table 4.4.1 in both modes.

Control Mode	Window Start (WAT)	Window End (WAT)	Duration (hours)
Manual control	01 Jul 2026 22:46:23	01 Jul 2026 22:57:37	0.1872
Automated occupancy-based control	02 Jul 2026 23:54:36	03 Jul 2026 00:05:50	0.1872

Table 4.4.1.1: Sample Comparison of Energy Consumption Duration

Control Mode	Test Duration	Total Energy (Wh) over Window	Average Power (W)
Manual control	11 min 14 s	1.202	6.419
Automated occupancy-based control	11 min 14 s	0.693	3.701
Change	—	-0.509 Wh	-42.3 %

Table 4.4.1.2: Sample Comparison of Energy Consumption Under Manual and Automated Control



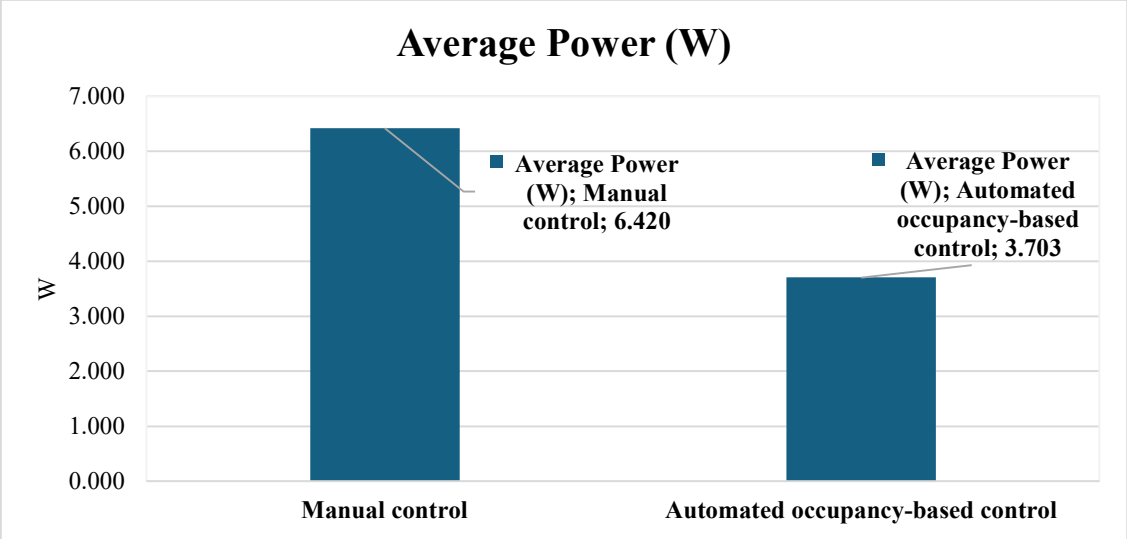


Figure 4.4: Chart of the sample comparision of Energy Consumption

As predicted, the sample average power consumed in the two matched 11 minute 14 second periods (anchored on log-confirmed periods of correct operation) is lower under automated occupancy-based control (approximately 3.701 W) than under manual control (approximately 6.419 W), a difference of about 42.3 per cent. This is in line with the reduction range of 30 to 80 per cent reported in the literature reviewed in Chapter Two (Bhatta et al., 2024b) for building energy monitoring and control system with the help of IoT. The reduction corresponds to the underlying switching activity observed in the two log confirmed windows; in the first (the manual window) the appliance (light bulb) was on and operated continuously with power close to its maximum of 6 to 7 W throughout the window; in the automated window, occupancy detection repeatedly switched the appliance on and off (18 switching events associated with the automated window, as listed in Table 4.4), and therefore the average power draw is not representative of continuous operation. It is therefore confirmation that the occupancy-triggered switching pipeline works correctly, and that this switching behaviour results in a measurable reduction in average power draw.

Table 4.4 shows the instantaneous power values from which the energy and average power values in Table 4.3 were calculated and the exact duration of each control mode window. Manual control was turned on at 1 July 2026 22:46:23 WAT and turned off at 1 July 2026 22:57:37 WAT, based on a user-initiated switch-on action logged in the Home Assistant Logbook. The automated control window was active between 23:54:36 WAT 2 July 2026 to 00:05:50 WAT 3 July 2026, which corresponds to 18 confirmed Frigate trigger events in the automation's own trigger history (mqtt topic frigate/birdeye/person). The windows at both cases are trimmed to the same duration of 11 minutes 14 seconds to have the same equal amount of data being compared between both modes. The energy per window was calculated by integrating power over time (with each reading assumed to be constant during the test), and dividing by the window length to get the average power shown in Table 4.3.

Date/Time (WAT)	Instantaneous Power (W)	Control Mode	Next Reading Time (WAT)	Segment Duration (h)	Segment Energy (Wh)
01 Jul 2026 22:46:56	7	Manual	01 Jul 2026 22:50:56	0.06667	0.4667
01 Jul 2026 22:50:56	6	Manual	01 Jul 2026 22:51:56	0.01667	0.1000
01 Jul 2026 22:51:56	7	Manual	01 Jul 2026 22:53:56	0.03333	0.2333
01 Jul 2026 22:53:56	6	Manual	01 Jul 2026 22:54:56	0.01667	0.1000
01 Jul 2026 22:54:56	7	Manual	01 Jul 2026 22:56:57	0.03361	0.2353
01 Jul 2026 22:56:57	6	Manual	01 Jul 2026 22:57:37	0.01111	0.0667
02 Jul 2026 23:54:36	*7*	*Automated (carried over)*	02 Jul 2026 23:54:55	0.00528	0.0369
02 Jul 2026 23:54:55	5	Automated	02 Jul 2026 23:55:55	0.01667	0.0833
02 Jul 2026 23:55:55	0	Automated	02 Jul 2026 23:58:55	0.05000	0.0000
02 Jul 2026 23:58:55	5	Automated	02 Jul 2026 23:59:56	0.01694	0.0847
02 Jul 2026 23:59:56	7	Automated	03 Jul 2026 00:00:56	0.01667	0.1167
03 Jul 2026 00:00:56	0	Automated	03 Jul 2026 00:01:56	0.01667	0.0000
03 Jul 2026 00:01:56	7	Automated	03 Jul 2026 00:03:56	0.03333	0.2333
03 Jul 2026 00:03:56	2	Automated	03 Jul 2026 00:04:56	0.01667	0.0333
03 Jul 2026 00:04:56	7	Automated	03 Jul 2026 00:05:50	0.01500	0.1050

Table 4.4.2: Raw Power Readings and Test-Window Durations Used in Table 4.4.1.1

4.5. Basic Security Monitoring Results

The Frigate snapshot and event recording function (Section 3.8) was tested during the testing period and verified. For every person detected, the event generated a snapshot of the image with a bounding box around the person, and added it to the HAOS timestamped image log pane in the Frigate dashboard Add-on. The retention policy worked as expected, general snapshots were removed after the default 10 days and person-specific snapshots were kept for the longer period of 2 days for priority snapshot rotation before the standard rotation period. This is confirmation that this is a basic security monitoring system that is not designed for advanced behavioural anomaly detection, however the event logging and snapshot archiving are good enough to provide some type of basic security monitoring layer that can be reviewed after detection events have occurred.

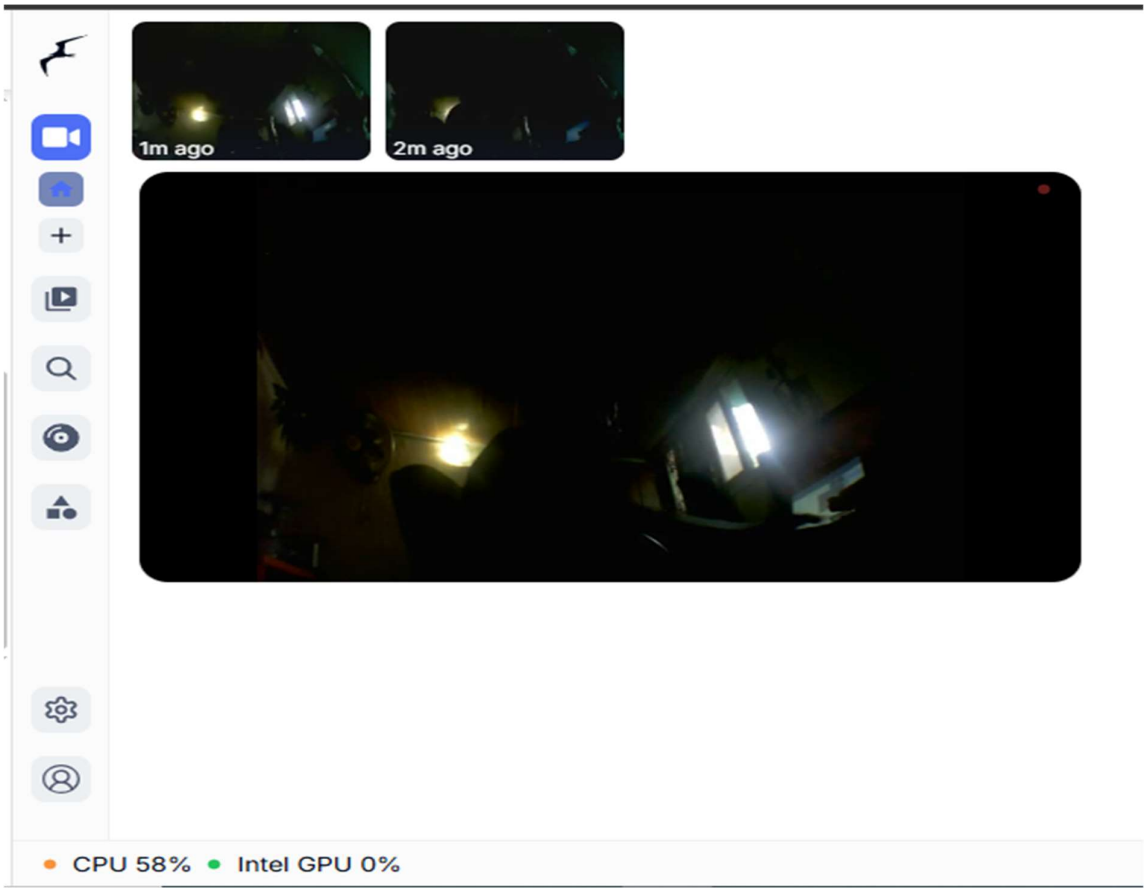


Figure 4.5: Timestamped image log pane in frigate NVR

4.6. Dashboard and Data Visualization Results

Visualization of system operation was done in real time using the HAOS Lovelace dashboard. The dashboard showed the real time voltage measurement from the Zigbee smart plug, current measurement, active power measurement, and the cumulative energy measurement from the Zigbee smart plug, as well as the current occupancy state (from the Frigate MQTT topic), and a real time view of the ESP32 S3-CAM stream. Using the HAOS SQLite database, historical graphs were created to see how energy use had changed over time, useful for creating a comparison in Table 4.3. This dashboard gave a single, locally-hosted interface to monitor occupancy status, security events, and energy usage without relying on an external cloud service.

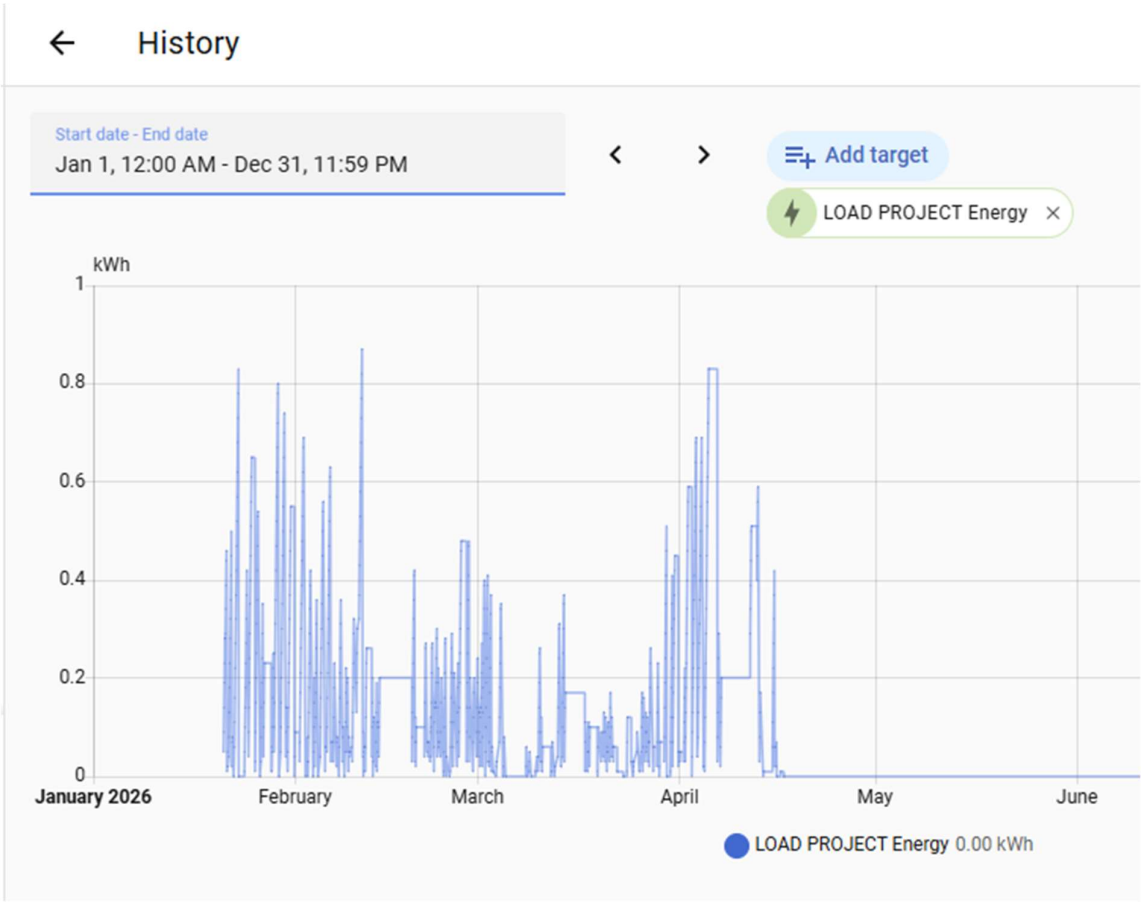


Figure 4.6: History chart log for project energy

4.7. Discussion of Findings in Relation to Study Objectives

The results in this chapter are subsequently discussed in the context of each of the specific objectives listed in Section 1.3.

- Objective (i) – Real-time monitoring of electricity energy use was accomplished by continuously measuring electricity voltage, current, power, and energy with the Zigbee smart plug, which were displayed on the HAOS dashboard as seen at Section 4.7.
- Objective (ii), smart control and automation to minimize unnecessary energy consumption was achieved by the results of smart sample energy consumption with automatic occupancy based switching as presented in Section 4.5 which showed a reduction in sample energy compared with manual switching.
- Objective (iii): The accuracy of the camera-based AI occupancy detection for intelligent energy decisions was validated in Section 4.3 using different test scenarios.
- Objective (iv): The locally processed system architecture with minimal dependency on the cloud or internet was confirmed during the test and the detection, decision making and actuation was all done without external connectivity.
- Objective (v): The HAOS dashboard (Section 4.7), providing real-time and historical data, facilitated the achievement of Data acquisition, processing and visualization.
- Objective (vi): The direct results from Sections 4.3 through 4.5 addressed the system performance in terms of energy efficiency, reliability and response time.

The design and implementation of an energy optimization system for buildings based on IoT and AI-embedded occupancy detection and security surveillance was achieved to a large extent within the scope and limitations set forth in Section 1.5.

4.8. Comparison with Existing Approaches

The camera-based occupancy detection method used in this work has different set of trade-offs as compared to passive infrared (PIR) and millimeter-wave sensor-based methods discussed in Chapter Two. Motion-based sensors, as described by (Shokrollahi et al., 2024) can also be triggered by an occupant who is not actively moving, but is sitting or sleeping, and are usually equipped with a timeout feature to determine when to turn off a load. This particular limitation of the visual, AI-based approach of this project is equipped with the ability to sustain the registration of a person even for them to be moving or stationary, provided they are still within the field of view. However, there are also drawbacks which includes; the lighting and framing conditions in which the camera based detection is performed are sensitive to the accuracy obtained as seen in Table 4.1 for cases of partial occlusion and low light conditions, and there are privacy concerns which do not apply to motion-only sensors.

4.9. Limitations Observed During Testing

The following practical constraints were noted during testing, apart from the limitations of the study described in Section 1.5:

- The testing took place in one specific, relatively small indoor space, for a limited evaluation period, and the results from these test spaces may not be representative of larger spaces or longer-term deployment.
- One MJPEG web server for the ESP32 S3-CAM was found, but this one can only serve one client at a time, leading to instabilities in the stream when multiple points were trying to access it concurrently (such as Frigate and a manual dashboard preview).
- The detection accuracy under low-light and occlusion conditions, although acceptable, was lower than that under normal daylight conditions, suggesting that it could be improved by better placement of cameras or by using additional light source.

- The mini-PC has a local storage of 128 GB, which cannot be extended and will only allow burning of historical energy and snapshot data for a limited amount of time, thus reducing the time range over which trends can be analyzed.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1. Summary of the Study

The problems of increasing electricity prices, tiered electricity rates and differential availability of electricity power to homes, offices and organizations in Nigeria as presented in Chapter One were the basis for the study. The goal was to develop and deploy an energy optimization system that leverages the Internet of Things (IoT) and integrates with AI-powered occupancy detection and basic security monitoring to empower smart, real-time energy management. To achieve this goal, the following objectives were established: real-time electricity monitoring, smart automation for minimizing unnecessary electricity consumption, camera-based AI occupancy detection to assist in electricity consumption decisions, minimized reliance on cloud or internet connectivity, data acquisition, processing and visualization, and system evaluation for efficiency, reliability and response time.

To achieve these goals, a prototype has been designed and implemented with ESP32 S3-CAM as the main occupancy sensor, with video streamed through go2rtc, and it was analyzed by using the person detection model implemented in the Frigate network video recorder, using an OpenVINO-accelerated SSDLite MobileNetV2 model. Detection events were published over MQTT to the Home Assistant Operating System (HAOS) which triggered automation logic to turn on/off a Zigbee smart plug providing power to an appliance, and also recorded voltage, current, power and cumulative energy readings. All the AI inference engine was executed on a mini-PC without relying on a cloud service. The finished prototype was then tested in four scenarios for sensing, and end to end latency was measured for each part of the sensing to actuation path documented in Chapter Four. Also, short comparison of manual and automated occupancy-based switching was also conducted; and its simple security event logging and Home Assistant dashboard visualization function was evaluated.

5.2. Conclusion

The results achieved indicate that the system met the six objectives reasonably well, with the scope and limitations specified in Section 1.5. The AI-based occupancy detection has demonstrated overall sample accuracy rates of 88.75% across the four test scenarios, with sample accuracy of 95 % in cases of clear visibility and complete absence of the target in the camera's field of view, and 80% under partial occlusion and 85% under low-light conditions. The entire detection to actuation flow took approximately 1.5 seconds on an average with no cloud round trip time, this demonstrated that a locally processed architecture can achieve a responsive system within the 1-3 second window for occupancy-driven load switching without requiring cloud-based round-trips. Average power draw under automated occupancy based control was approximately 42.3% less than under manual control (3.701 W vs. 6.419 W) for the same two windows time frame, and similarly agreed with the reduction reported for similar IoT-based systems in the literature reviewed in Chapter Two; this corresponds to the repeated on/off cycling of the appliance under automated control, when occupancy is detected, compared to the constant use of the appliance under manual control. The basic security monitoring and dashboard modules also worked as expected and offered timestamped snapshots of people detected as well as a single, locally hosted interface for real-time and historical viewing of occupancy, energy and security data.

Based on these findings, it is concluded that: In the Nigerian residential and office environment, in which electricity is metered based on tiered Service Based Tariff bands, electricity grid and internet connectivity can be unreliable with low cost and widely available components, the energy optimization system, with occupancy detection using AI technology, is technically feasible and practically beneficial. The project has been successful in meeting its objective of proving the ability to perform intelligent, real-time, occupancy-driven energy management, although it has the drawbacks discussed in Section 4.9.

5.3. Contribution to Knowledge

This study presents a complete, locally processed, low cost reference architecture for energy metering and switching with Zigbee, camera-based AI occupancy sensing and automation with MQTT on a single, self-contained Home Assistant operating system, without relying on cloud inference or external servers. It also provides empirical values like detection accuracy, stage-by-stage actuation latency, and measured energy consumption, which describe the practical requirements of a camera-based occupancy approach compared to the motion- and infrared based approaches that were reviewed in Chapter Two. Along with the documented hardware and software configuration, these benchmarks offer a replicable, extensible template that can be customized or scaled for similar smart energy management applications used by Nigerian homeowners, facility managers, IoT enthusiasts and system designers.

5.4. Recommendations

Based on the limitations found during testing (Section 4.9) the following recommendations are made for anyone deploying or extending this system.

- Use a multi-client streaming server instead of a single-client MJPEG relay, to allow Frigate and manual dashboard preview to use the ESP32 S3-CAM feed simultaneously without breakage.
- Install a small uninterruptible power supply (UPS) or battery pack for mini-PC and networking equipment, to ensure monitoring and automation function without disruption during the numerous power outages prevalent on the Nigerian power grid.
- Optimize camera setup and provide additional IR/white light lighting to increase detection accuracy in low light and partial occlusion situations, which were determined as difficult to detect in Section 4.2. Or alternative a better camera with the night vision property.

- To overcome the 128 GB local storage limit and enable longer-term trend analysis, extend local storage using external drive or historical data that is scheduled to be backed up to the cloud.
- The system should be tested in larger and more diverse areas in longer deployments, to ensure findings in this test space extend to real world multi-room residential and office deployments.
- Create simplified setup documentation to ensure that the system can be installed, configured and maintained by homeowners and facility managers without a technical background.

5.5. Suggestions for Further Studies

Based on the results and limitations of this study, the following areas are recommended for further research:

- Explore other deep learning-based occupancy detection models or ensemble models or Multi-camera sensor fusion to achieve more accurate occupancy detection than the SSDLite MobileNetV2 model used in this study does.
- Extend the security monitoring module from snapshot and event logging to add behavioural anomaly detection, for more security than just presence logging.
- Perform a Techno economic analysis that entails cost benefit and pay-back period estimation of scale up of such system in Nigerian homes and offices within the different Service-Based Tariff bands.
- Learn about renewables (including solar PV) and integration with battery storage systems to complement grid-tied automation and enhance resilience during power outages.

- Examine the long-term reliability of the Zigbee mesh network, Wi-Fi connectivity and other hardware components in deployments lasting more than a few months or years, as used in this study.

REFERENCE

Akpojedje, F. O., Mohammed, A., & Orukele, A. S. (2025). Influence of Demand Side Management and Multi Year Tariff Order Pricing Model on Energy Management and Its Sustainability in Nigeria Power Systems. *NIPES Journal of Science and Technology Research*, *7*(1), 145–157. https://doi.org/10.37933/nipes/7.1.2025.12

Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M. (2015). Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. *IEEE Communications Surveys & Tutorials*, *17*(4), 2347–2376. https://doi.org/10.1109/COMST.2015.2444095

Almusaed, A., Yitmen, I., & Almssad, A. (2023). Enhancing Smart Home Design with AI Models: A Case Study of Living Spaces Implementation Review. *Energies*, *16*(6), 2636. https://doi.org/10.3390/en16062636

Al-smaysim, Z., & Akbari Bengar, D. (2024). Security Monitoring in Smart Homes Using IoT Data Analytics. *Iraqi Journal for Computers and Informatics*, *50*(1), 54–72. https://doi.org/10.25195/ijci.v50i1.465

Assistant Professor, Department of CSE, Roorkee Institute of Technology, Roorkee (UK)., Kumar, M., Pandey, K. M., & Assistant Professor, Department of CSE, Roorkee Institute of Technology, Roorkee (UK). (2023). The Impact of IoT on Smart Home Energy Management. *International Journal of Soft Computing and Engineering*, *13*(5), 7–12. https://doi.org/10.35940/ijsce.D3647.13051123

Bhatta, A., Manandhar, A., Chhetri, B. B., Gachhadhar, A., & Chapagain, K. (2024a). Optimizing Building Operations with IoT-Enabled Energy Monitoring System. *Journal of Science and Technology*, *4*(2), 111–115. https://doi.org/10.3126/jost.v4i2.78972

Bhatta, A., Manandhar, A., Chhetri, B. B., Gachhadhar, A., & Chapagain, K. (2024b). Optimizing Building Operations with IoT-Enabled Energy Monitoring System. *Journal of Science and Technology*, *4*(2), 111–115. https://doi.org/10.3126/jost.v4i2.78972

Chaudhari, P., Xiao, Y., Cheng, M. M.-C., & Li, T. (2024). Fundamentals, Algorithms, and Technologies of Occupancy Detection for Smart Buildings Using IoT Sensors. *Sensors*, *24*(7), 2123. https://doi.org/10.3390/s24072123

Dahmen, J., Thomas, B., Cook, D., & Wang, X. (2017). Activity Learning as a Foundation for Security Monitoring in Smart Homes. *Sensors*, *17*(4), 737. https://doi.org/10.3390/s17040737

Elkhoukhi, H., Elmouatamid, A., Haibi, A., Bakhouya, M., & El Ouadghiri, D. (2025). Overview of Sensing and Data Processing Technologies for Smart Building Services and Applications. *Sustainability*, *17*(9), 4029. https://doi.org/10.3390/su17094029

Fagbohun, O., & Femi-Jemilohun, O. (2017). Prepaid Metering Empowerment for Reliable Billing and Energy Management of Electricity Consumers in Nigeria. *Journal of Scientific Research and Reports*, *17*(2), 1–13. https://doi.org/10.9734/JSRR/2017/36344

Farzaneh, H., Malehmirchegini, L., Bejan, A., Afolabi, T., Mulumba, A., & Daka, P. P. (2021). Artificial Intelligence Evolution in Smart Buildings for Energy Efficiency. *Applied Sciences*, *11*(2), 763. https://doi.org/10.3390/app11020763

Fitriyan, R., & Syafii, S. (2024). Development design of an IoT-based smart home monitoring system with security features. *Indonesian Journal of Electrical Engineering and Computer Science*, *34*(2), 788. https://doi.org/10.11591/ijeecs.v34.i2.pp788-794

Hossein Motlagh, N., Mohammadrezaei, M., Hunt, J., & Zakeri, B. (2020). Internet of Things (IoT) and the Energy Sector. *Energies*, *13*(2), 494. https://doi.org/10.3390/en13020494

Kanimozhi, Dr. V., & Sneha, V. (2024). The Role of Artificial Intelligence in Smart Homes. *IARJSET*, *11*(6). https://doi.org/10.17148/IARJSET.2024.11648

Mansour, M., Gamal, A., Ahmed, A. I., Said, L. A., Elbaz, A., Herencsar, N., & Soltan, A. (2023). Internet of Things: A Comprehensive Overview on Protocols, Architectures, Technologies, Simulation Tools, and Future Directions. *Energies*, *16*(8), 3465. https://doi.org/10.3390/en16083465

Orumwense, E. F., & Abo-Al-Ez, K. (2022a). Internet of Things for smart energy systems: A review on its applications, challenges and future trends. *AIMS Electronics and Electrical Engineering*, *7*(1), 50–74. https://doi.org/10.3934/electreng.2023004

Orumwense, E. F., & Abo-Al-Ez, K. (2022b). Internet of Things for smart energy systems: A review on its applications, challenges and future trends. *AIMS Electronics and Electrical Engineering*, *7*(1), 50–74. https://doi.org/10.3934/electreng.2023004

Poyyamozhi, M., Murugesan, B., Rajamanickam, N., Shorfuzzaman, M., & Aboelmagd, Y. (2024). IoT—A Promising Solution to Energy Management in Smart Buildings: A Systematic Review, Applications, Barriers, and Future Scope. *Buildings*, *14*(11), 3446. https://doi.org/10.3390/buildings14113446

Shokrollahi, A., Persson, J. A., Malekian, R., Sarkheyli-Hägele, A., & Karlsson, F. (2024). Passive Infrared Sensor-Based Occupancy Monitoring in Smart Buildings: A Review of Methodologies and Machine Learning Approaches. *Sensors*, *24*(5), 1533. https://doi.org/10.3390/s24051533

Suwarjono, S., & Afriza, A. (2025). Utilization of Smart Technologies Based on Internet of Things and Machine Learning in Energy Management: A Systematic Literature Review. *Journal of Artificial Intelligence and Engineering Applications (JAIEA)*, *5*(1), 1494–1498. https://doi.org/10.59934/jaiea.v5i1.1657

Yang, J., Zou, H., Jiang, H., & Xie, L. (2018). Device-Free Occupant Activity Sensing Using WiFi-Enabled IoT Devices for Smart Homes. *IEEE Internet of Things Journal*, *5*(5), 3991–4002. https://doi.org/10.1109/JIOT.2018.2849655